DÜZCE UNIVERSITY | FACULTY OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING

2025-2026 ACADEMIC

YEAR SPRING

SEMESTER

CE498 - COMPUTER ENGINEERING PROJECT DESIGN

Supervisor:

Dr. Büşra TAKGİL

AN ARTIFICIAL INTELLIGENCE–SUPPORTED EXPENSE AND INVOICE ANALYSIS SYSTEM

Prepared by:

Uğur AVŞAR

Student No:211015040

BM401 BİLGİSAYAR MÜHENDİSLİĞİ PROJE TASARIMI / BM498 MEZUNİYET TEZİ DEĞERLENDİRME VE SÖZLÜ SINAV TUTANAĞI		
ÖĞRENCİLER (NUMARA/AD-SOYAD):		
DANIŞMAN:		
Değerlendirme Kriteri	**Puan Aralığı**	**Puan**
Yazılı Çalışma Biçimsel Değerlendirme (10 Puan)		
Çalışma kılavuza uygun olarak hazırlanmış mı?	0-10	
Problemin tanımı açıkça yapılmış mı?	0-5	
Literatür taraması yeterli, güncel ve sistematik bir yaklaşımla (PRISMA vb.) gerçekleştirilmiş mi?	0-5	
Geliştirilecek yazılımın/donanımın mimarisini içeren blok şeması (yazılımlar için veri akış şeması da olabilir) çizilerek açıklanmış mı?	0-5	
Kullanılan materyal ve metot yeterli düzeyde anlatılmış mı?	0-5	
Projede kullanılan veri setleri FAIR (Findable, Accessible, Interoperable, Reusable) ilkelerine uygun şekilde yönetilmiş mi?	0-5	
Tasarımın uygulamasında ortaya çıkan uyumsuzluklar ve aksaklıklar belirtilerek çözüm yöntemleri tartışılmış mı?	0-5	
Yapılan işlerin zorluk derecesi?	0-5	
Yapılan sunum başarılı mı?	0-8	
Soruları yanıtlama yetkinliği?	0-7	
Öğrenci dönem içerisindeki raporlarını düzenli olarak hazırladı mı?	0-10	
Etik ve Mesleki Sorumluluk (15 Puan)		
Proje etik ilkelere uygun mu? (İntihal, etik beyan, veri ve yapay zekâ kullanımı, insan verisi kullanımı – Helsinki Deklarasyonu vb.). ☐ İntihal beyanı mevcut ☐ ISO 690 uygun kaynak gösterimi yapılmış ☐ İnsan verisi kullanılmışsa etik izin / onam mevcut ☐ Helsinki Deklarasyonu ilkelerine uygun	0-5	
Projede mühendislik etiği ve mesleki sorumluluk bilinciyle hareket edilmiş mi?	0-5	
Mühendislik uygulamalarında ulusal/uluslararası standartlar belirtilmiş (ISO 9001, ISO 17025, IEEE) ve uygulanmış mı?	0-5	
Proje sürecinde proje yönetimi, risk yönetimi ve değişiklik yönetimi yaklaşımları uygulanmış mı? (İş planı, görev dağılımı, risk analizi)	0-4	
Girişimcilik ve yenilikçilik boyutu değerlendirilmiş mi? (Özgün çözüm, katma değer, ticari potansiyel)	0-4	
Sürdürülebilir kalkınma perspektifi (çevresel, ekonomik ve sosyal sürdürülebilirlik) gözetilmiş mi?	0-4	
Sürdürülebilir Yazılım Geliştirmede Yeniden Üretilebilirlik (Reproducibility) ilkesi uygulanmış mı?	0-3	
Toplam	0-100	

DECLARATION

I hereby declare that this thesis is my own work, that there has been no unethical conduct at any stage from the planning to the writing of the thesis, that all information in this thesis has been obtained within the framework of academic and ethical rules, that all information and interpretations not obtained through this thesis work have been properly cited with their sources and these sources have been included in the references list, and that no patent or copyright infringement has occurred during the study and writing of this thesis.

01 June 2026

Uğur Avşar

GENERATIVE ARTIFICIAL INTELLIGENCE USAGE DECLARATION

I hereby declare that I have received support from **Claude (Anthropic), ChatGPT (OpenAI), and Google Gemini** generative artificial intelligence programs during the preparation of this thesis. During the preparation of my thesis, I received support from generative artificial intelligence programs (e.g., language translation, access to scientific articles, text editing). I confirm that I have verified the accuracy of the information obtained from generative artificial intelligence programs.

01 June 2026

Uğur Avşar

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my esteemed supervisor, **Dr. Büşra Takgil**, for her support, guidance, and valuable contributions throughout my undergraduate education and thesis study.

I would also like to thank my beloved family, whose moral and material support I have always felt throughout this process, and my friends for their understanding and support.

Furthermore, I would like to express my gratitude to all my instructors who contributed to my undergraduate education and to everyone who contributed to the completion of this study.

01 June 2026

Uğur Avşar

CONTENT

2025-2026 ACADEMIC YEAR SPRING SEMESTER ..... **i**
DECLARATION ..... **iii**
GENERATIVE ARTIFICIAL INTELLIGENCE USAGE DECLARATION ..... **iv**
ACKNOWLEDGEMENTS ..... **v**
CONTENT ..... **vi**
FIGURE CONTENT ..... **ix**
TABLE CONTENT ..... **x**
ABBREVIATIONS ..... **xi**
ABSTRACT ..... **xii**
AN ARTIFICAL INTELLIGENCE–SUPPORTED EXPENSE AND INVOICE ANALYSIS SYSTEM xii
1. INTRODUCTION **1**
1.1. Problem Definition 1
1.2. Aim and Importance of the Study 1
1.3. Research Questions 2
1.4 Scope and Limitations 4
1.4.1. Scope of the Study 4
1.4.2. Limitations of the Study 4
1.5. Structure of the Thesis 5
2. LITERATURE REVIEW **6**
2.1. Manual Expense Tracking and Its Limitations 6
2.2. Mobile-Based Expense Tracking Applications 6
2.3. Expense Categorization Systems 7
2.4. OCR and Receipt Scanning Technologies 9
2.5. Large Language Models for Document and Invoice Analysis 10
2.6. System Architecture and Security 11
2.7. Summary and Positioning of This Study 12
3. TOOLS AND METHODS **16**
3.1. System Requirements Analysis 16
3.1.1. Functional Requirements 16
3.1.2. Non-Functional Requirements 17

3.1.3. Development Methodology 18
3.2. System Architecture 19
3.2.1. Overall Architecture Overview 19
3.2.2. Layered Architectural Approach 19
3.2.3. AI Microservice Architecture 20
3.3. Database Design 20
3.3.1. Users Table 21
3.3.2. Expenses Table 21
3.3.3. Budget Limits Table 22
3.4. Backend Implementation 23
3.4.1. Technology Stack 23
3.4.2. REST API Endpoints 24
3.4.3. Authentication and Security Implementation 25
3.5. AI Microservice Implementation 25
3.5.1. Google Gemini 2.5 Flash Integration 25
3.5.2. Invoice Photo Analysis 26
3.5.3. CSV and XLSX File Analysis 26
3.5.4. Statistical AI Analysis 26
3.6. Mobile Application Implementation 27
3.6.1. Technology Stack 27
3.6.2. Authentication Screens 27
3.6.3. Home Screen 31
3.6.4. Expense Entry and Editing 32
3.6.5. AI-Assisted Import Screen 32
3.6.6. Statistics Screen 34
3.6.7. Profile Screen 37
4. FINDINGS AND DISCUSSION 39
4.1. Testing Strategy 39
4.2. Backend API Testing 39
4.2.1. Authentication and Email Verification Testing 39
4.2.2. Expense Management Endpoint Testing 40
4.2.3. AI Microservice Integration Testing 41
4.2.4. Security and Authorization Testing 42

4.3. Mobile Application Testing 42
4.3.1. Authentication Flow Testing 42
4.3.2. Expense Management Testing 43
4.3.3. AI-Assisted Import Testing 43
4.3.4. Statistics and Budget Testing 43
4.3.5. Profile and Settings Testing 44
4.4. User Scenario Validation 44
4.5. Comparison with Literature 45
4.6. Limitations 46
5. CONCLUSION AND FUTURE WORK 48
5.1. Conclusion 48
6. REFERANCES 52
CURRICULUM VITAE xiii

FIGURE CONTENT

Figure 3.1.Login and registration screens of the FaturaAnaliz mobile application 29
Figure 3.2. Email verification message and confirmation screen 29
Figure 3.3.Password reset screen and OTP email delivered to the user 30
Figure 3.4.OTP entry and successful password reset confirmation screens 30
Figure 3.5.Home screen displaying monthly total, category distribution chart, and recent expenses 31
Figure 3.6.Expense entry screen showing amount input, category selection dropdown, and edit/delete actions 32
Figure 3.7.AI-assisted import screen showing CSV/XLSX file import and photo import tabs 33
Figure 3.8.Invoice photo analysis result displaying extracted amount of 892.25 TL and predicted category. 34
Figure 3.9.AI analysis tab displaying Gemini-generated commentary, anomaly detection, and risk assessment. 36
Figure 3.10.Budget management tab showing category budget goal dialog and color-coded spending progress indicators. 36
Figure 3.11.Monthly comparison tab displaying AI-generated commentary and category-level percentage changes 37
Figure 3.12.Profile screen in light and dark modes showing user statistics and application settings. 38
Figure 3.13.Language selection dialog and English-localized profile screen. 38
Figure 4.1.Successful authentication test via Swagger UI showing JWT generation. 40
Figure 4.2.Successful expense creation test demonstrating user-based data isolation via Swagger UI. 41
Figure 4.3.Authorization test showing rejection of unauthorized access without a valid JWT token via Swagger UI. 42

TABLE CONTENT

TABLE 2.1.SUMMARY TABLE OF THE LİTERATURE REVİEW 13
TABLE 3.1.FİELDS AND DESCRİPTİONS OF THE USERS DATABASE TABLE 21
TABLE 3.2.FİELDS AND DESCRİPTİONS OF THE EXPENSES DATABASE TABLE..... 22
TABLE 3.3.FİELDS AND DESCRİPTİONS OF THE BUDGET_LİMİTS DATABASE TABLE..... 22
TABLE 3.4.SUMMARİZES THE RESTFUL API ENDPOİNTS THAT ENABLE AUTHENTİCATİON, EXPENSE MANAGEMENT, BUDGETİNG, REPORTİNG, AND AI-BASED ANALYSİS WİTHİN THE SYSTEM 24

ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
APK	Android Package Kit
BCrypt	Blowfish Crypt (password hashing algorithm)
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
OCR	Optical Character Recognition
ORM	Object-Relational Mapping
OTP	One-Time Password
PDF	Portable Document Format
REST	Representational State Transfer
SDK	Software Development Kit
SQL	Structured Query Language
UI	User Interface

ABSTRACT

AN ARTIFICAL INTELLIGENCE–SUPPORTED EXPENSE AND INVOICE ANALYSIS SYSTEM

Uğur AVŞAR

Düzce University

Faculty of Engineering, Computer Engineering

Undergraduate Thesis

Supervisor: Dr. Büşra TAKGİL

June 2026, 54 pages

In this study, a mobile-based expense and invoice management system was developed that enables users to securely record, manage, and analyze their personal financial data with artificial intelligence support. The system allows users to register and log in securely, add expenses manually, view and delete existing expenses, and analyze spending data through category-based filtering and graphical reports. Developed using a layered client–server architecture, the backend is secured with Java Spring Boot and JWT-based authentication, the mobile application is built with the Flutter framework to support cross-platform deployment, and data is stored per user in a PostgreSQL relational database. Google Gemini 2.5 Flash API was integrated as the artificial intelligence module, enabling automatic analysis of invoice photographs, category prediction during bulk expense import from CSV/XLSX files, and natural language commentary generation based on monthly spending data. Additional features include user-defined budget limits per category, monthly comparative statistics, and PDF report generation. Tests conducted throughout the development process demonstrated that the system is functional, secure, and extensible.

Keywords: Expense Tracking, Mobile Application, Personal Finance Management, Artificial Intelligence, Google Gemini, Spring Boot, Flutter, JWT

1. INTRODUCTION

1.1. Problem Definition

Tracking daily expenses in a consistent and structured manner has become increasingly important for individuals seeking to manage their personal finances effectively. In practice, however, this process tends to be fragmented and unsustainable. Physical receipts accumulate without being recorded, invoice photographs get lost among unrelated files, and digital payment records are scattered across different applications and email inboxes. This disorganized structure makes it difficult to construct a coherent financial overview and evaluate spending habits over time.

Existing applications address this gap only partially. Tools such as Mint, Wallet, and Money Manager offer basic categorization and reporting features, yet their underlying logic typically relies on fixed rule-based systems that struggle to adapt to diverse user profiles and varying invoice formats. Moreover, the majority of these systems operate exclusively on text-based transaction data, which means that visual financial documents such as photographed receipts and invoices cannot be processed automatically.

Academic research on AI-assisted expense categorization has similarly remained within the boundaries of conventional machine learning methods, including Naive Bayes classifiers and Support Vector Machines. The integration of multimodal AI models — capable of jointly processing both image and text inputs — into this domain has received limited attention in the literature. Furthermore, in most existing implementations, AI components are tightly coupled with the core system backend, which significantly reduces modularity and makes it difficult to update or replace the AI module independently.

Taken together, these shortcomings highlight a clear need for a mobile-based expense management system that can process invoice images, perform automated categorization, and be built upon a modular and maintainable architecture.

1.2. Aim and Importance of the Study

The primary aim of this study is to design and develop a mobile-based expense management system that integrates the Google Gemini 2.5 Flash API as a multimodal AI engine for automated financial data extraction and expense categorization. Unlike conventional expense

tracking applications that require users to manually enter all financial information, the proposed system allows users to submit invoice images directly through the mobile application and receive automatically generated expense records in return. This approach transforms the user's role from active data entry operator to passive reviewer, substantially reducing the effort required for consistent financial tracking.

The system is designed as a unified digital platform that supports secure registration and authentication, expense entry through either manual input or AI-assisted invoice analysis, category-based filtering, and graphical statistical reporting. The Flutter-based mobile client targets Android devices, while the Spring Boot backend manages all business logic and data operations through a RESTful API. AI analysis is handled by an independently deployed Python microservice that communicates with the Gemini API, processes submitted invoice images, and returns structured financial data to the backend.

A secondary architectural aim is to demonstrate a loosely coupled integration model in which the AI component is isolated as an independent microservice. This design ensures that model updates or prompt modifications can be applied without requiring changes to the core backend application, improving long-term maintainability.

The importance of this study can be evaluated across three dimensions. From a practical standpoint, it directly addresses the manual data entry dependency that limits existing personal finance applications. From a technical standpoint, it provides a concrete implementation of Gemini 2.5 Flash within a full-stack mobile application. From an academic standpoint, it contributes empirical evidence to a notably underexplored area concerning the use of vision-language models for invoice image analysis in a mobile context.

1.3. Research Questions

The design and development activities conducted within the scope of this study are guided by a set of clearly defined research questions. These questions address both the technical feasibility of the proposed system and the practical effectiveness of its AI integration, and are directly reflected in the evaluation activities presented in Chapter 4.

RQ1: Can the Google Gemini 2.5 Flash API accurately extract structured financial information from invoice images submitted through a mobile application?

Invoice images encountered in real-world usage vary considerably in terms of layout, language, font style, print quality, and handwritten content. This question investigates whether the Gemini

2.5 Flash model can reliably identify and extract key financial fields — specifically the total expense amount, transaction date, and merchant or service description — across this diversity of formats. Extraction accuracy is a fundamental prerequisite for the usefulness of the AI-assisted expense entry feature.

RQ2: Does AI-based automatic expense categorization meaningfully reduce the manual input burden on users?

This question examines the degree to which the model's category predictions align with the correct expense categories defined within the system, and evaluates whether the model assigns categories consistently across different spending domains including food, transportation, health, education, and entertainment. The answer determines whether users can reasonably accept AI-generated categories without manual correction in the majority of cases.

RQ3: Does the decoupled Python microservice architecture allow AI integration without negatively affecting system stability and response times?

This question investigates whether the additional communication overhead introduced by the microservice approach causes measurable performance degradation in standard expense management operations, and whether AI processing latency remains within acceptable bounds for a mobile application.

RQ4: How does the system perform across end-to-end user scenarios that combine AI-assisted invoice analysis with manual expense management and statistical reporting?

This question addresses the complete workflow — from a user capturing an invoice image and receiving AI-extracted data, to confirming the record and viewing updated statistics on the reporting screen. End-to-end testing is essential for identifying integration issues not apparent in component-level tests and for validating a coherent user experience across all system layers.

These four research questions collectively cover the technical, architectural, and experiential dimensions of the developed system. The findings obtained in response to each question are presented and discussed in Chapter 4, with reference to relevant prior work identified in Chapter 2.

1.4 Scope and Limitations

1.4.1. Scope of the Study

The scope of this study encompasses the end-to-end design, implementation, and evaluation of a mobile-based expense and invoice management system with integrated multimodal AI capabilities. The system consists of four primary components: a Flutter-based mobile client application, a Spring Boot REST API backend, a PostgreSQL relational database, and a Python-based AI microservice connected to the Google Gemini 2.5 Flash API.

The following functional capabilities are fully designed, implemented, and evaluated within the scope of this study:

- **User authentication and authorization:** Secure registration and login through JWT-based stateless authentication with per-user data isolation.
- **Manual expense entry:** Structured input of expense amount, category, and description through a dedicated form interface.
- **AI-assisted invoice analysis:** Automatic extraction of expense amount, transaction date, and category from invoice images processed by the Gemini 2.5 Flash model.
- **Expense management:** Chronological listing and deletion of recorded expenses.
- **Category-based filtering:** Retrieval and display of expenses belonging to a selected category through a dedicated filtering screen.
- **Statistical reporting:** Graphical presentation of total expenditure, daily average, category distribution, and monthly spending trends.
- **System testing and evaluation:** Structured assessment of backend functionality, mobile application behavior, and AI categorization accuracy.

1.4.2. Limitations of the Study

Despite the breadth of functionality implemented, several limitations define the boundaries of the current system and the conclusions that can be drawn from its evaluation.

Dependency on external AI service: The AI-assisted invoice analysis feature is entirely dependent on the availability and response quality of the Google Gemini 2.5 Flash API. No offline fallback or locally hosted model is implemented. In the event of API unavailability, the AI feature becomes non-functional and users must revert to manual entry.

Limited invoice format coverage: AI accuracy is evaluated using a manually prepared test set that covers common invoice formats but does not fully represent the diversity of invoice types across different industries, countries, and languages.

Single currency support: The system supports single-currency expense tracking only, with no multi-currency conversion or aggregation functionality. Users managing expenses across different currencies would require a customized extension of the current system.

Manual category corrections not tracked: When a user modifies an AI-generated category prediction before saving, only the final selection is stored. The original prediction is not retained, making it impossible to retrospectively compute correction rates or leverage user feedback for improving categorization behavior.

1.5. Structure of the Thesis

The remainder of this thesis is organized as follows. Chapter 2 presents a comprehensive review of the related literature, examining prior studies on mobile-based expense tracking systems, AI-driven financial data classification, and the application of large language models to document and invoice analysis. Chapter 3 describes the tools and methods employed in the development of the system, covering the overall system architecture, the technology stack, the database design, the backend API implementation, the Google Gemini 2.5 Flash microservice integration, and the mobile application development process. Chapter 4 presents the findings obtained from functional testing, security evaluation, and AI categorization accuracy assessment, and discusses these findings in relation to the research questions defined in Chapter 1 and the existing literature reviewed in Chapter 2. Chapter 5 concludes the thesis with a general evaluation of the study outcomes, a discussion of the system's real-world implications, and recommendations for future research and development directions.

2. LITERATURE REVIEW

2.1. Manual Expense Tracking and Its Limitations

Personal financial management has been a widely studied domain due to its direct impact on individuals' economic well-being and long-term financial stability. A recurring finding across the literature is that manual methods for recording income and expenditures, while accessible, are fundamentally limited in their sustainability and reliability. Sabab et al. demonstrated through the development of the eExpense system that the core problem motivating digital expense tracking solutions is the difficulty individuals face when attempting to record and analyze financial transactions consistently over time [1]. Their study identified that users who rely on manual recording methods frequently encounter data loss, inconsistency across recorded entries, and an inability to generate meaningful financial summaries from fragmented records.

Further evidence of the inadequacy of manual approaches is provided by studies examining the behavioral patterns of expense tracking application users. Ismail et al. confirm that users require structured digital tools not only for recording transactions but also for organizing and visualizing financial data in ways that support budget awareness [2]. Without automated data organization, users are left to interpret raw lists of transactions, a task that provides limited actionable insight into their spending behavior. These foundational observations establish the core motivation for intelligent, automated expense management systems and underline the importance of reducing manual data entry as a primary design objective.

2.2. Mobile-Based Expense Tracking Applications

The widespread adoption of smartphones as primary computing devices has positioned mobile applications as the most practical platform for personal expense management. Garcia and Claour developed the Mobile Bookkeeper application, a personal financial management system that integrates a receipt scanner powered by Optical Character Recognition to enable users to capture expense data from physical receipts through their smartphone camera [3]. Their evaluation, conducted using ISO 25062 and ISO 9241 usability standards, established receipt scanning as a highly valued feature that addressed a critical gap in existing finance applications, namely the absence of automated data capture from physical documents. This study provided

an early demonstration of the feasibility and user acceptance of camera-based expense data entry in a mobile context.

The Voicordit system introduced an alternative input modality, demonstrating that voice-to-text conversion powered by deep learning models including Convolutional Neural Networks and Long Short-Term Memory networks could be used to record daily expenditures through spoken commands [4]. Their system showed that reducing the friction of data entry, regardless of the input method used, significantly improved users' willingness to record expenses consistently. This finding reinforces the broader principle that the primary barrier to sustained expense tracking is the cognitive and physical effort required for data entry, rather than any lack of user motivation.

Sharma et al. reviewed AI-powered personal finance management applications and found that the integration of machine learning, natural language processing, and predictive analytics within mobile finance systems substantially improves user engagement and financial decision-making quality [5]. Their analysis highlighted that users of AI-integrated finance applications demonstrated higher rates of budget adherence and more structured long-term financial planning compared to users of conventional manual-entry applications. However, they also noted that technical complexity and privacy concerns remained significant barriers to the widespread adoption of AI-integrated finance tools.

The Money Map system presented at the 2025 ICKECS conference demonstrated a data-driven personal finance tracker that leverages machine learning to analyze spending patterns, forecast budgets, and generate personalized financial insights through dynamic data visualization [6]. This system represented one of the more comprehensive recent implementations of an intelligent personal finance mobile application, incorporating multilingual support and data encryption alongside its analytical capabilities. Its architecture demonstrated the growing expectation among users and researchers alike that modern personal finance applications should provide not only data storage but also intelligent interpretation of financial behavior.

2.3. Expense Categorization Systems

The organization of financial transactions into meaningful categories is widely recognized as a prerequisite for effective personal financial analysis. The WONGA system, presented at the 2023 IEEE INCET conference, proposed a machine learning-driven approach to personal finance management that automatically categorizes each extracted expense into the appropriate category [7]. The system further generated customized monthly budget plans based on

identified spending patterns, demonstrating that reliable automated categorization enables downstream personalization features that would be impractical to implement in manually-organized systems. WONGA's reliance on SMS message parsing for transaction data capture highlighted both the potential and the limitations of text-based extraction approaches, which perform well for standardized banking messages but struggle with the diversity of formats encountered in real-world expense records.

Shaik et al. conducted a systematic exploration of automatic expense tracking systems and found that machine learning-based categorization models consistently outperform traditional rule-based approaches in accurately handling the diversity of transaction types encountered in real-world usage [8]. Their research demonstrated that user satisfaction with automated categorization was closely tied to prediction accuracy, with systems achieving above 85% category accuracy receiving significantly higher user ratings than those operating below this threshold. This finding establishes a concrete accuracy benchmark against which AI-based categorization systems, including the one developed in this thesis, can be evaluated.

Traditional approaches to automated categorization have predominantly employed Support Vector Machine classifiers applied to short text descriptions of financial transactions. García-Méndez et al. developed a novel system combining natural language processing techniques with SVM classification to identify and categorize banking transaction descriptions for personal finance management [9]. Their system used character and word n-gram features to represent short financial text messages and achieved competitive classification performance on a curated labeled corpus. However, the authors acknowledged that their approach required substantial manually labeled training data to generalize across the diversity of merchant names and transaction description formats encountered in real banking environments, pointing toward the need for more data-efficient categorization approaches.

More scalable categorization approaches have been subsequently developed. Toran et al. proposed a weakly supervised framework for bank transaction classification that combines weak supervision signals with deep neural network architectures, demonstrating that this approach can outperform market-leading commercial categorization solutions while requiring significantly less manually labeled training data [10]. This line of research highlighted the fundamental tension between classification accuracy and annotation cost that characterizes automated financial categorization, and suggested that semi-supervised and weakly supervised

approaches represent a more practical path toward scalable categorization than fully supervised methods.

The application of Large Language Models to financial text classification has been explored by Pavlinek et al., who examined resource-efficient few-shot LLM-based approaches for banking transaction categorization [11]. Their findings demonstrated that LLM-based classification could achieve competitive performance with minimal labeled examples, opening the possibility of deploying categorization capabilities in data-limited domains without the extensive annotation pipelines required by traditional supervised learning approaches. This work is directly relevant to the AI-assisted categorization implemented in the current study, which employs the Gemini 2.5 Flash model in a zero-shot prompting configuration to predict expense categories from invoice image content.

2.4. OCR and Receipt Scanning Technologies

Optical Character Recognition has long been recognized as a key enabling technology for automating the extraction of structured financial data from physical and digital receipts and invoices. Dela Cruz et al. developed a mobile expense tracking application using Tesseract OCR version 5, which implements a Long Short-Term Memory-based recognition engine [12]. Their evaluation on twenty grocery and market receipts reported an average Character Error Rate of 9.36% and a Word Error Rate of 19.39% for item descriptions, with significantly lower error rates of 2.26% CER and 4.33% WER for unit price fields. These results demonstrated that while Tesseract OCR v5 provides functional text extraction for structured receipt fields such as prices and totals, its accuracy on free-form item descriptions remains a limiting factor for fully automated expense categorization based on OCR output alone.

Ismail et al. demonstrated that automated receipt scanning approaches based on Convolutional Recurrent Neural Networks achieve improved performance over traditional OCR pipelines, particularly for receipts with diverse font styles, variable text orientations, and partially occluded content [13]. By jointly performing text detection and recognition within a single end-to-end neural architecture, CRNN-based systems can leverage contextual information across the full receipt image to improve extraction accuracy on fields that would be challenging for character-level recognition systems operating in isolation.

The ReceiptSense dataset and benchmark introduced by Abdallah et al. provided the research community with a standardized evaluation resource for receipt understanding that extends beyond simple text extraction to encompass the semantic relationships between detected fields

[14]. Their dataset comprises 20,000 annotated receipts, 30,000 OCR-annotated images, and a Receipt QA subset pairing receipt images with 40 question-answer pairs each, enabling the evaluation of models on tasks that require understanding the relationships between item names, quantities, prices, and totals. Their benchmark results revealed that while OCR accuracy on individual fields was high for modern systems, correctly associating related fields across complex receipt layouts remained a significant open challenge, motivating the shift toward holistic document understanding models.

2.5. Large Language Models for Document and Invoice Analysis

The introduction of large language models with multimodal capabilities has fundamentally transformed the landscape of document understanding and information extraction. The Gemini family of models, introduced by Google DeepMind, demonstrated state-of-the-art performance across 30 of 32 multimodal benchmarks, establishing that vision-language models trained at scale could process and understand document images with a level of semantic comprehension substantially exceeding previous OCR-based approaches [15]. The Gemini architecture's ability to jointly process image and text inputs within a unified transformer-based framework enables a qualitatively different approach to invoice analysis compared to traditional pipeline-based systems, where OCR, layout analysis, and information extraction are handled by separate specialized models.

Gemini 1.5 extended these capabilities by introducing support for extremely long context windows, enabling the model to process and reason over multi-page documents within a single inference call [16]. This architectural advancement has direct implications for invoice analysis, as it allows the model to consider the full document context when extracting specific fields rather than processing isolated text regions independently, substantially improving accuracy on documents where relevant information is distributed across multiple sections or pages.

The most recent Gemini 2.5 family further advances these capabilities with significantly improved reasoning, stronger multimodal understanding, and enhanced performance on structured data extraction tasks [17]. Gemini 2.5 Flash, the variant employed in this study, is specifically optimized for high-throughput, low-latency inference, making it well-suited for real-time invoice processing within mobile application workflows where response latency directly impacts user experience.

Berghaus et al. conducted a comprehensive benchmark of eight multimodal large language models including models from the Gemini 2.5 family on three diverse invoice document

datasets using zero-shot prompting [18]. Their results demonstrated that native image processing generally outperforms text-based structured parsing approaches, with Gemini 2.5 achieving competitive or superior performance compared to GPT-based alternatives on structured extraction tasks. These findings directly inform the architectural decision in the present study to use direct image submission to the Gemini API rather than a multi-stage pipeline involving OCR preprocessing.

Kovács et al. presented an LLM-centric pipeline for information extraction from invoices at the IEEE FLLM2024 conference, providing a structured analysis of architectural patterns for integrating LLM-based extraction into document processing workflows [19]. Their work identified the decoupled microservice pattern as particularly suitable for production deployments, where the AI component must remain independently scalable, replaceable, and maintainable without requiring modifications to the core application backend. This architectural recommendation is directly reflected in the Python microservice design adopted in the present study.

Gómez and Sánchez evaluated the capability of general-purpose large language models to extract structured information from electricity invoices without task-specific fine-tuning [20]. Their study benchmarked Gemini 1.5 Pro and Mistral-small across 19 parameter configurations and six prompting strategies, finding that prompt quality was the dominant factor in extraction accuracy, with the best few-shot strategy exceeding the zero-shot baseline by more than 19 percentage points in F1 score. This finding underlines the importance of prompt engineering in LLM-based invoice extraction systems and informed the structured prompt design employed in the Gemini API integration developed in this thesis.

2.6. System Architecture and Security

The architectural design of backend systems supporting mobile applications has been studied extensively in the context of RESTful API design, microservice decomposition, and authentication security. Li et al. demonstrated that JWT-based stateless authentication provides a scalable and performant security mechanism for multi-user mobile applications, enabling user identity verification without server-side session storage [21]. This property is particularly valuable in horizontally scalable backend deployments where maintaining session state across

multiple server instances would introduce significant architectural complexity and reduce system resilience.

Pangarkar et al. presented research on enhancing the performance and scalability of a Flutter-based application through a microservices architecture deployed on AWS [22]. Their study demonstrated that decomposing application functionality into independently deployable microservices, combined with Docker containerization and load balancing, enables mobile backends to handle high-traffic loads while maintaining response time consistency. The Flutter framework's cross-platform capabilities were confirmed to enable consistent behavior across different mobile platforms when communicating with a well-designed REST API layer, providing relevant architectural validation for the Flutter and Spring Boot combination adopted in the present study.

2.7. Summary and Positioning of This Study

The reviewed literature establishes a clear progression in the field of personal expense management, from manual recording methods, through mobile application-based digital tracking with OCR and rule-based categorization, to AI-assisted automated systems leveraging machine learning and, most recently, large language models. Table 2.1 summarizes the primary contributions and limitations of the key studies reviewed in this chapter.

The present study is positioned at the current frontier of this progression, integrating the Google Gemini 2.5 Flash API to perform both image-based data extraction and expense category prediction within a single inference step. This approach eliminates the sequential OCR-then-classify pipeline adopted by prior systems and replaces it with a holistic multimodal analysis that handles text extraction, layout understanding, and semantic categorization simultaneously. The decoupled Python microservice architecture adopted for AI integration addresses the maintainability limitations identified in prior embedded AI implementations, and the full-stack mobile application context provides an evaluation environment that goes beyond the isolated model assessments that characterize most prior LLM invoice extraction studies.

Table 2.1.Summary Table of the Literature Review

Authors and Year	Aim of the Study	Technologies and Methods Used	Results and Findings
Sabab et al. (2018)	To identify the core problem motivating digital expense tracking solutions	eExpense system; mobile application development; user behavior analysis	Manual recording methods lead to data loss and inconsistency; the need for structured digital tools was confirmed
Garcia and Claour (2021)	To evaluate the feasibility of OCR-assisted receipt scanning in a mobile context.	Optical Character Recognition; ISO 25062 and ISO 9241 usability standards.	Camera-based data entry demonstrated high user acceptance and addressed a critical gap in existing applications
Santos et al. (2024)	To examine the effect of voice-based expense recording on sustained tracking behavior	CNN and LSTM-based voice-to-text conversion; deep learning	Reducing data entry friction significantly improved consistent recording behavior
Sharma et al. (2025)	To investigate the impact of AI-powered personal finance applications on user behavior	Machine learning; natural language processing; predictive analytics	AI-integrated applications produced higher budget adherence rates and more structured long-term financial planning
Shailendra et al. (2025)	To develop and evaluate a data-driven personal finance tracking system	Machine learning; dynamic data visualization; multilingual support	Modern personal finance applications must support behavioral interpretation, not only data storage
M. I. R et al. (2023)	To propose machine learning-based automatic categorization for personal finance management	SMS message parsing; machine learning classification; budget planning	Automated categorization enabled personalized budget planning; limitations of text-based approaches were identified
Shaik et al. (2024)	To conduct a systematic exploration of automatic expense tracking systems	Machine learning-based classification; comparison with rule-based systems	Category accuracy above 85% was significantly associated with higher user satisfaction rates
García-Méndez et al. (2020)	To classify banking transaction descriptions using SVM	NLP; SVM classification; character and word n-gram features	Competitive classification performance was achieved; large labeled datasets are required for broad generalization

Toran et al. (2023)	To propose a weakly supervised framework for bank transaction classification	Weak supervision; deep neural network architectures	Performance surpassing commercial solutions was achieved with significantly less labeled training data
Pavlinek et al. (2023)	To examine LLM-based few-shot approaches for banking transaction	Large language models; few-shot learning; text classificationclassification; expense description analysis.	LLM-based classification achieved competitive performance with minimal labeled examples
Dela Cruz et al. (2023)	To develop a mobile receipt-based expense tracker using Tesseract OCR v5	Tesseract OCR v5; LSTM-based recognition engine	Low error rates were achieved for price fields; limitations were identified for free-form item descriptions
Ismail et al. (2024)	To evaluate CRNN-based automated receipt scanning performance	Convolutional Recurrent Neural Network; end-to-end architecture	Improved performance over traditional OCR pipelines was demonstrated
Abdallah et al. (2024)	To provide a standardized benchmark resource for receipt understanding	20,000 annotated receipts; OCR evaluation; Receipt QA subset	Field-level OCR accuracy is high; relational field association across complex layouts remains an open challenge
Gemini Team (2023)	To introduce the Gemini multimodal model family	Unified transformer architecture; image and text processing	State-of-the-art performance was achieved on 30 of 32 multimodal benchmarks
Gemini Team (2024)	To introduce Gemini 1.5 with extended context window support	Long-context processing; multi-page document understanding	Processing of multi-page documents within a single inference call was made possible
Gemini Team (2025)	To introduce Gemini 2.5 with improved reasoning and multimodal capabilities	Enhanced reasoning; structured data extraction; low-latency inference	Optimized for high-throughput inference; well-suited for real-time mobile application workflows
Berghaus et al. (2025)	To benchmark multiple LLMs on invoice processing tasks	Zero-shot prompting; multi-model comparison; diverse invoice datasets	Native image processing outperformed text-based parsing; Gemini 2.5 achieved competitive performance

Kovács et al. (2024)	To analyze LLM-centric pipeline architecture for invoice information extraction	LLM integration architecture; microservice pattern; document processing	The decoupled microservice pattern was identified as the most suitable architectural approach for production deployments
Gómez and Sánchez (2026)	To evaluate general-purpose LLMs for invoice information extraction without fine-tuning	Gemini 1.5 Pro; Mistral-small; six prompting strategies	Prompt quality was the dominant performance factor; the best few-shot strategy exceeded the zero-shot baseline by over 19 percentage points
Li et al. (2020)	To investigate JWT-based authentication in microservice architecture	JWT; stateless authentication; microservice security	JWT-based authentication provides a scalable and performant security mechanism for multi-user mobile applications
Pangarkar et al. (2024)	To improve the performance and scalability of a Flutter-based application through microservices	Flutter; microservices; AWS; Docker containerization	Independent microservices maintained consistent response times under high traffic; Flutter's cross-platform capabilities were confirmed

3. TOOLS AND METHODS

This chapter presents the tools, technologies, and methods employed in the design and implementation of the developed mobile-based expense and invoice management system. The chapter begins with a requirements analysis defining the functional and non-functional constraints of the system, followed by a description of the adopted development methodology. Subsequent sections detail the system architecture, database design, backend implementation, AI microservice integration, and mobile application development. Each section describes not only what was implemented but also the rationale behind the selection of specific technologies and architectural decisions.

3.1. System Requirements Analysis

The requirements analysis phase of the developed system was conducted with the goal of identifying both the functional capabilities that the system must provide to end users and the non-functional quality attributes that must be satisfied across all system components. This analysis informed all subsequent design and implementation decisions and served as the primary basis for evaluating system completeness during the testing phase.

3.1.1. Functional Requirements

The functional requirements of the system define the specific behaviors and operations that the application must support. These requirements were identified based on the core use cases of personal expense management and AI-assisted invoice analysis, and are organized as follows.

Authentication and Account Management: Users must be able to create new accounts by providing their full name, Gmail address, password, and password confirmation. Upon successful registration, the system must send an email verification link to the provided Gmail address; account activation must require successful completion of email verification before login is permitted. Users must be able to log in using their verified Gmail address and password. Users must be able to initiate a password reset process by providing their registered Gmail address, receiving a six-digit one-time password via email, and submitting the code alongside a new password to complete the reset. Users must be able to log out of the application through an explicit confirmation dialog.

Expense Management: Users must be able to manually add expense records by entering an amount, an optional description, and selecting a category from a predefined list. Users must be able to view their complete expense history in a chronological list on the home screen, with

each record displaying its category icon, name, category label, and amount. Users must be able to edit individual expense records after creation and delete them through a context menu accessible from the expense list.

AI-Assisted Import: Users must be able to import expenses from CSV or XLSX files; the system must use the Google Gemini 2.5 Flash API to automatically predict the expense category for each record based on the description field. Users must be able to capture invoice or receipt images using the device camera or select existing images from the device gallery; the system must submit these images to the Gemini 2.5 Flash API for automated extraction of the total amount and category prediction, and present the extracted data to the user for review before saving.

Statistics and Reporting: The system must display a summary screen showing the current month's total expense amount, daily average expenditure, and most frequently occurring expense category. The system must present a donut chart visualizing the proportional distribution of expenses across active categories and a bar chart displaying spending trends over the last seven days. The system must provide an AI Analysis tab that presents AI-generated commentary on the user's spending behavior, identifies statistically anomalous expense entries, assesses overall spending risk level, and provides AI-generated saving recommendations. The system must provide a Budget tab allowing users to define daily, weekly, and monthly spending limits for each expense category independently, and display real-time progress bars comparing current spending against the defined limits. The system must provide a Monthly tab comparing the current month's total expenditure against the previous month's total, presenting AI-generated commentary on category-level changes, and displaying a side-by-side category comparison table.

Profile and Settings: The system must display a user profile screen showing the user's name, email address, total expense amount, daily average, and active category count. Users must be able to toggle between light and dark visual themes. Users must be able to switch the application language between Turkish and English. Users must be able to download a monthly expense report as a PDF document. Users must be able to permanently delete all their expense records through a confirmation-protected action.

3.1.2. Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of the system, independent of specific feature behaviors.

Security: All API endpoints that access user-specific data must require a valid JSON Web Token in the Authorization header. Passwords must be stored in hashed form using BCrypt. Email verification must be enforced before account activation. Password reset codes must expire after fifteen minutes.

Usability: The mobile application interface must be accessible to users without technical expertise. All primary actions must be reachable within two navigation steps from the home screen. Input validation feedback must be displayed immediately upon form submission failure.

Performance: API responses for standard expense management operations must be returned within acceptable latency bounds for a mobile application context. The AI analysis microservice must return Gemini API results within a timeframe that does not disrupt the user's workflow.

Scalability: The system architecture must support the independent scaling of the AI microservice without requiring modifications to the core Spring Boot backend.

Maintainability: The AI microservice must be replaceable or upgradeable independently of the backend. The Flutter application must support both Turkish and English localization without requiring separate builds.

3.1.3. Development Methodology

The system was developed following an iterative and incremental development approach. Rather than attempting to implement all system features in a single development cycle, the implementation was organized into successive iterations each targeting a specific functional layer of the system. The first iteration focused on establishing the backend infrastructure, database schema, and JWT-based authentication mechanism. The second iteration implemented the core expense management endpoints and the basic mobile client interface. The third iteration introduced the AI microservice and the Gemini 2.5 Flash integration for invoice photo analysis and CSV import. The fourth and final iteration added the statistics screen with its four tabs, the budget management module, the profile screen with dark mode and language support, and the monthly PDF report feature.

This iterative approach allowed each layer of the system to be tested and validated before the next layer was added, reducing the risk of integration failures and ensuring that foundational components such as authentication and data persistence were stable before more complex AI-dependent features were built on top of them.

3.2. System Architecture

3.2.1. Overall Architecture Overview

The developed system follows a layered client-server architecture consisting of four independently operating components: a Flutter-based mobile client, a Spring Boot REST API backend, a PostgreSQL relational database, and a Python-based AI microservice connected to the Google Gemini 2.5 Flash API. Each component operates within a clearly defined boundary of responsibility, and all inter-component communication is conducted over HTTP using structured JSON data exchange.

The mobile client application serves as the primary user interface and is responsible for rendering all application screens, managing local state, handling user interactions, and communicating with the backend API. The Spring Boot backend handles all business logic, user authentication, data persistence coordination, and acts as the orchestration layer between the mobile client and the AI microservice. The PostgreSQL database provides persistent relational storage for user accounts, expense records, and budget limits. The Python microservice functions as a dedicated AI processing layer, receiving requests from the Spring Boot backend, communicating with the Google Gemini 2.5 Flash API, and returning structured analysis results.

3.2.2. Layered Architectural Approach

The Spring Boot backend is organized according to a multi-layer architecture in which each layer is responsible for a specific set of concerns.

The **Controller Layer** contains RESTful endpoint definitions that handle incoming HTTP requests from the mobile client. This layer is responsible solely for request routing, input deserialization, and response serialization. No business logic is implemented at the controller level; all incoming requests are forwarded to the appropriate service component.

The **Service Layer** contains the core business logic of the application. Operations including user registration with email verification, authentication with JWT issuance, expense creation, modification and deletion, category-based filtering, statistical aggregation, budget management, and AI microservice invocation are all implemented within this layer.

The **Repository Layer** provides the data access interface between the service layer and the PostgreSQL database, implemented using Spring Data JPA. Each repository interface extends JpaRepository and provides typed query methods for its corresponding database entity.

The **Security Layer** is implemented using Spring Security and is responsible for JWT-based request authentication. Every incoming request is intercepted by a JWT filter that validates the token, extracts the user identity, and populates the security context before the request reaches the controller layer.

3.2.3. AI Microservice Architecture

The AI microservice is implemented as an independent Python service built with the Flask framework. It exposes dedicated endpoints that receive requests from the Spring Boot backend and return structured JSON responses. The microservice supports three distinct analysis modes: invoice image analysis, CSV/XLSX batch expense analysis, and statistical spending analysis for AI commentary generation.

For invoice image analysis, the microservice receives a base64-encoded image, constructs a structured prompt instructing the Gemini 2.5 Flash model to extract the total amount and predict the most appropriate expense category, submits the image and prompt to the Gemini API, and parses the returned JSON response before forwarding the extracted fields to the backend.

For CSV/XLSX analysis, the microservice reads the uploaded file, extracts description and amount fields, and for each record submits the description text to the Gemini API with a classification prompt to predict the appropriate category from the predefined category set.

For statistical AI analysis, the microservice receives the user's aggregated expense data, constructs a comprehensive prompt providing category totals, daily averages, and anomaly indicators, and requests the Gemini model to generate natural language commentary, risk assessments, and saving recommendations.

3.3. Database Design

The system uses PostgreSQL as its relational database management system. PostgreSQL was selected over alternative relational database options for several reasons. Its robust support for ACID transactions ensures data integrity across concurrent multi-user operations. Its native support for JSON data types provides flexibility for storing structured AI response metadata if required in future extensions. Its seamless integration with the Spring Data JPA and Hibernate ORM stack eliminates the need for manual query construction and simplifies the data access layer implementation. Additionally, PostgreSQL's mature indexing capabilities and proven performance at production scale make it a reliable choice for a system designed with scalability as a non-functional requirement.

The database is structured around three primary tables: **users**, **expenses**, and **budget_limits**, with relational constraints enforced through foreign key references.

3.3.1. Users Table

The **users** table stores all registered user account information. Its fields are defined as follows:

Table 3.1.Fields and descriptions of the users database table

Field	Type	Description
id	BIGINT (PK)	Auto-generated unique user identifier
full_name	VARCHAR	User's full name provided at registration
email	VARCHAR (UNIQUE)	Email address used for authentication
password	VARCHAR	BCrypt-hashed password
is_verified	BOOLEAN	Email verification status; login blocked if false
verification_token	VARCHAR	UUID token sent in verification email
reset_code	VARCHAR	6-digit OTP for password reset
reset_code_expiry	TIMESTAMP	Expiration timestamp for reset code (15 minutes)
created_at	TIMESTAMP	Account creation timestamp

3.3.2. Expenses Table

The **expenses** table stores all expense records associated with registered users. Its fields are defined as follows:

Table 3.2.Fields and descriptions of the expenses database table

Field	Type	Description
id	BIGINT (PK)	Auto-generated unique expense identifier
amount	DECIMAL	Expense amount in Turkish Lira
description	VARCHAR	Optional user-provided description
category	VARCHAR	Expense category from predefined set
created_at	TIMESTAMP	Date and time of expense entryv
user_id	BIGINT (FK)	Foreign key referencing users.id

3.3.3. Budget Limits Table

The **budget_limits** table stores per-category spending targets defined by each user. A separate record is maintained for each user-category combination, allowing independent budget configuration across all ten expense categories. Its fields are defined as follows:

Table 3.3.Fields and descriptions of the budget_limits database table

Field	Type	Description
id	BIGINT (PK)	Auto-generated unique record identifier
user_id	BIGINT (FK)	Foreign key referencing users.id
category	VARCHAR	Expense category to which the limits apply
daily_limit	DECIMAL	User-defined daily spending limit for this category

weekly_limit	DECIMAL	User-defined weekly spending limit for this category
monthly_limit	DECIMAL	User-defined monthly spending limit for this category
updated_at	TIMESTAMP	Timestamp of the most recent update to this record

The predefined category set consists of ten values: Market, Yeme & İçme, Faturalar, Ulaşım, Kira & Ev, Sağlık, Eğlence, Giyim, Eğitim, and Diğer. This set is used consistently across manual expense entry, AI-assisted categorization, budget tracking, and statistical reporting throughout all layers of the application.

3.4. Backend Implementation

3.4.1. Technology Stack

The backend application is developed using the **Spring Boot 3** framework with Java. Spring Boot was selected for its mature ecosystem, built-in support for RESTful API development, seamless integration with Spring Security and Spring Data JPA, and its production-ready embedded Tomcat server capabilities that simplify deployment without requiring external application server configuration. Compared to alternative backend frameworks such as Node.js with Express or Django with Django REST Framework, Spring Boot offers a more opinionated and structured approach to enterprise application development, with stronger compile-time type safety and a richer built-in security infrastructure through Spring Security. These properties make it particularly well-suited for a multi-user financial application where security correctness and maintainability are primary concerns.

BCrypt is used for password hashing through Spring Security's PasswordEncoder interface. **JavaMailSender** is used for sending HTML-formatted verification and password reset emails. **JWT tokens** are issued upon successful authentication and validated on every subsequent API request using the jjwt library.

3.4.2. REST API Endpoints

The backend exposes the following RESTful endpoints. All endpoints except those under /api/auth/** require a valid Bearer JWT token in the Authorization header.

Table 3.4.Summarizes the RESTful API endpoints that enable authentication, expense management, budgeting, reporting, and AI-based analysis within the system

Method	Endpoint	Description
POST	/api/auth/register	Register new user account
GET	/api/auth/verify-email	Activate account via email token
POST	/api/auth/login	Authenticate and receive JWT
POST	/api/auth/forgot-password	Send 6-digit OTP to email
POST	/api/auth/reset-password	Reset password using OTP
GET	/api/expenses	List all expenses for authenticated user
POST	/api/expenses	Create new expense record
PUT	/api/expenses/{id}	Update existing expense record
DELETE	/api/expenses/{id}	Delete expense record
GET	/api/expenses/stats	Get aggregated statistics for current user
POST	/api/import/photo	Submit invoice image for AI analysis
POST	/api/import/file	Submit CSV/XLSX file for AI analysis
GET	/api/budget	Get budget limits for current user
POST	/api/budget	Save or update budget limits
GET	/api/report/monthly	Generate and download monthly PDF report

3.4.3. Authentication and Security Implementation

The authentication flow begins at the registration endpoint. When a user submits a registration request, the backend validates that the provided email address is not already registered, hashes the supplied password using BCrypt, generates a UUID verification token, persists the user record with is_verified set to false, and dispatches an HTML verification email containing a unique activation link valid for twenty-four hours.

Upon clicking the verification link, the backend locates the user record by the token, sets is_verified to true, clears the token field, and returns a web confirmation page confirming successful account activation.

The login endpoint validates the provided credentials against the stored BCrypt hash, confirms that the account has been verified, and upon success generates and returns a JWT containing the user's ID and email as claims, signed with a server-side secret key.

The password reset flow is initiated by submitting a registered email address to the forgot-password endpoint. The backend generates a random six-digit numeric OTP, stores it alongside a fifteen-minute expiration timestamp in the user record, and dispatches an HTML-formatted email containing the code. The reset-password endpoint validates the submitted code against the stored value, confirms the code has not expired, hashes the new password, and clears the OTP fields from the database upon successful completion.

3.5. AI Microservice Implementation

3.5.1. Google Gemini 2.5 Flash Integration

The Google Gemini 2.5 Flash model is accessed through the official Google Generative AI Python SDK. The model is configured with a structured system prompt that instructs it to respond exclusively with valid JSON objects containing the expected field names. This response format constraint is essential for reliable parsing of model outputs within the automated pipeline and prevents the model from generating explanatory text that would require additional parsing logic.

Gemini 2.5 Flash was selected over other available models for three primary reasons. First, its optimized inference latency makes it well-suited for real-time invoice analysis within a mobile application context where response delay directly affects user experience. Second, its multimodal capability allows both image and text inputs to be processed within a single API call, eliminating the need for a separate OCR preprocessing stage. Third, its strong instruction-

following behavior on structured output tasks ensures consistent JSON response format across diverse invoice types and text inputs.

3.5.2. Invoice Photo Analysis

When a user selects the Fotoğraf tab within the İçe Aktar screen and captures or selects an invoice image, the following sequence of operations is executed. The Flutter application encodes the selected image in base64 format and transmits it to the Spring Boot backend via a multipart HTTP POST request. The backend forwards the image data to the Python microservice's /analyze-image endpoint. The microservice decodes the base64 image, constructs a multimodal prompt containing both the image content and a structured instruction requesting extraction of the total amount and category prediction, submits the combined input to the Gemini 2.5 Flash API, and parses the returned JSON response. The extracted amount and predicted category are returned to the backend and subsequently to the mobile client. The user is presented with the extracted data in a preview card showing the predicted category label and amount, along with a summary indicating the total number of records analyzed, the number successfully processed, and the number skipped. The user can then confirm the results and save the record with a single tap.

3.5.3. CSV and XLSX File Analysis

The Dosya tab of the İçe Aktar screen supports batch expense import from CSV and XLSX files. The file must contain at minimum a tutar (amount) column and an aciklama (description) column. The microservice reads the file using the pandas library, iterates over each row, and for each record submits the description text to the Gemini 2.5 Flash API with a category classification prompt. The model selects the most appropriate category from the predefined ten-category set based on the semantic content of the description. The microservice aggregates the category predictions and returns a structured list of expense objects to the backend for batch persistence.

3.5.4. Statistical AI Analysis

The AI Analiz and Aylık tabs of the statistics screen are powered by the statistical analysis endpoint of the microservice. The Spring Boot backend aggregates the authenticated user's expense data, computing category totals, the daily expenditure average, and identifying statistically anomalous entries defined as individual expense amounts exceeding two standard deviations above the user's mean expense amount. This aggregated data is transmitted to the microservice's /analyze-stats endpoint as a structured JSON payload. The microservice

constructs a detailed prompt containing the spending summary and requests the Gemini model to generate Turkish-language commentary covering the highest spending category, identified anomalies, an overall risk assessment level, specific category-level recommendations, and an estimated monthly saving potential. The generated commentary is parsed and returned as structured fields that the mobile client renders across the AI Analiz and Aylık tabs.

3.6. Mobile Application Implementation

3.6.1. Technology Stack

The mobile application is developed using the **Flutter** framework with the Dart programming language. Flutter was selected over alternative cross-platform frameworks such as React Native and over native Android development with Kotlin for several reasons. Unlike React Native, which renders through a JavaScript bridge to native components and can introduce performance inconsistencies across platforms, Flutter uses its own high-performance rendering engine built on Skia and, in newer versions, Impeller, which produces pixel-identical results on all supported platforms without relying on platform-native UI components. This property is particularly valuable for an application with custom chart visualizations and theme-switching functionality, where rendering consistency is important for a predictable user experience. Compared to native Kotlin development, Flutter enables simultaneous targeting of Android and potential future iOS deployment from a single codebase, reducing long-term maintenance overhead.

The application communicates with the Spring Boot backend exclusively through HTTP requests managed by the http package. Local state management is handled using Flutter's built-in setState and Provider patterns. Chart visualizations are implemented using the fl_chart package. PDF report generation is handled via the pdf and printing packages. Internationalization supporting Turkish and English is implemented through Flutter's intl and localization delegate system. The image_picker package is used for camera and gallery access in the invoice import feature.

3.6.2. Authentication Screens

The login screen presents the application logo, a Gmail address input field, a password field with a visibility toggle, a "Şifremi Unuttum" link, a "Giriş Yap" primary action button, and a "Kayıt Ol" navigation link. Input validation is performed client-side before the login request is dispatched. Success responses from the backend result in JWT storage and navigation to the home screen.

The registration screen collects the user's full name, Gmail address, password, and password confirmation. Client-side validation confirms that the password and confirmation fields match before the registration request is submitted. Upon successful registration, the user is informed that a verification email has been sent to their address. The verification email contains an "Email Adresimi Doğrula" button linking to the backend verification endpoint and includes the verification token in plain text as a fallback. The web confirmation page displayed after successful verification confirms account activation and instructs the user to return to the application.

The forgotten password screen collects the user's Gmail address and triggers dispatch of a six-digit OTP to that address upon submission. The password reset email presents the code prominently in a styled card with a note indicating its fifteen-minute validity period. The reset screen accepts the code alongside a new password and confirmation, submits the reset request to the backend, and upon success navigates back to the login screen with a "Şifreniz başarıyla güncellendi!" confirmation toast.

Figure 3.1., Figure 3.2., Figure 3.3. and Figure 3.4. shows the complete authentication flow of the mobile application, including the login and registration screens, the email verification email and web confirmation page, the forgotten password screen, the OTP email, the password reset screen, and the success notification.

Figure 3.1.Login and registration screens of the FaturaAnaliz mobile application

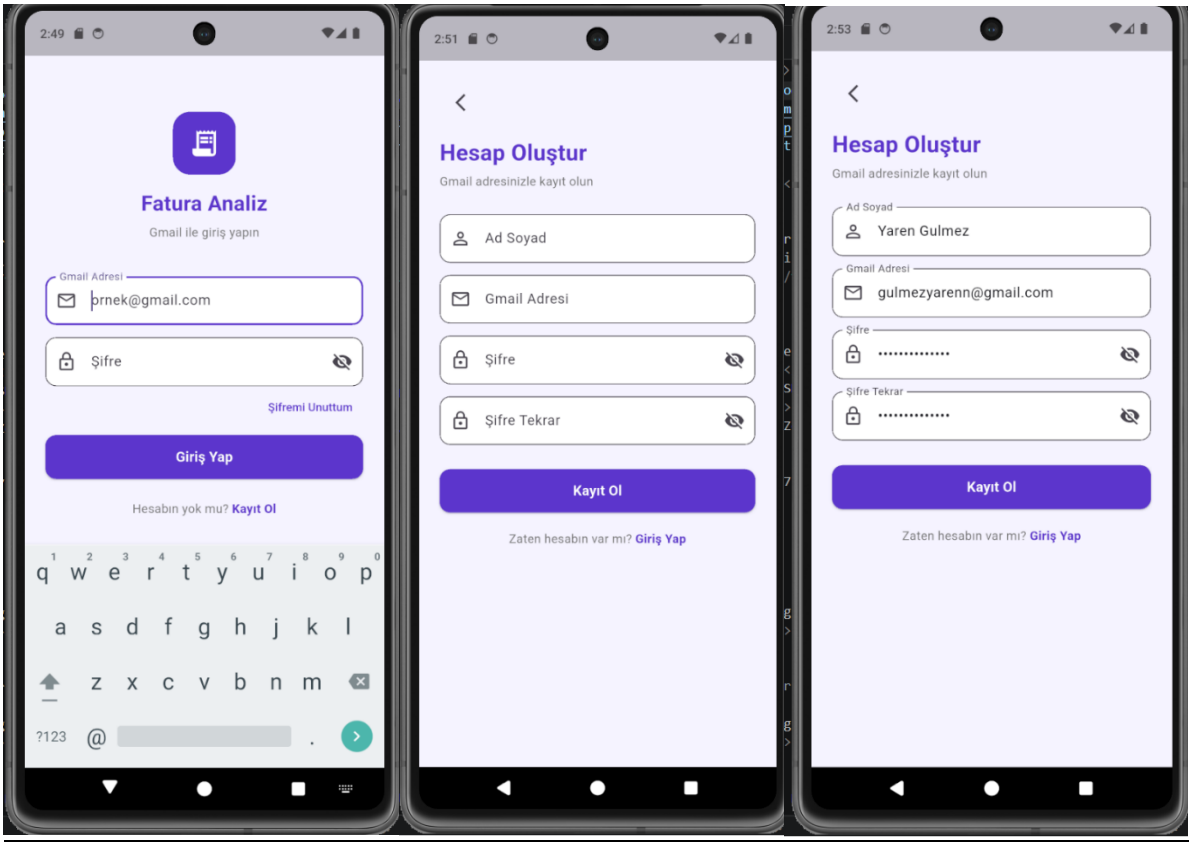


Figure 3.2. Email verification message and confirmation screen

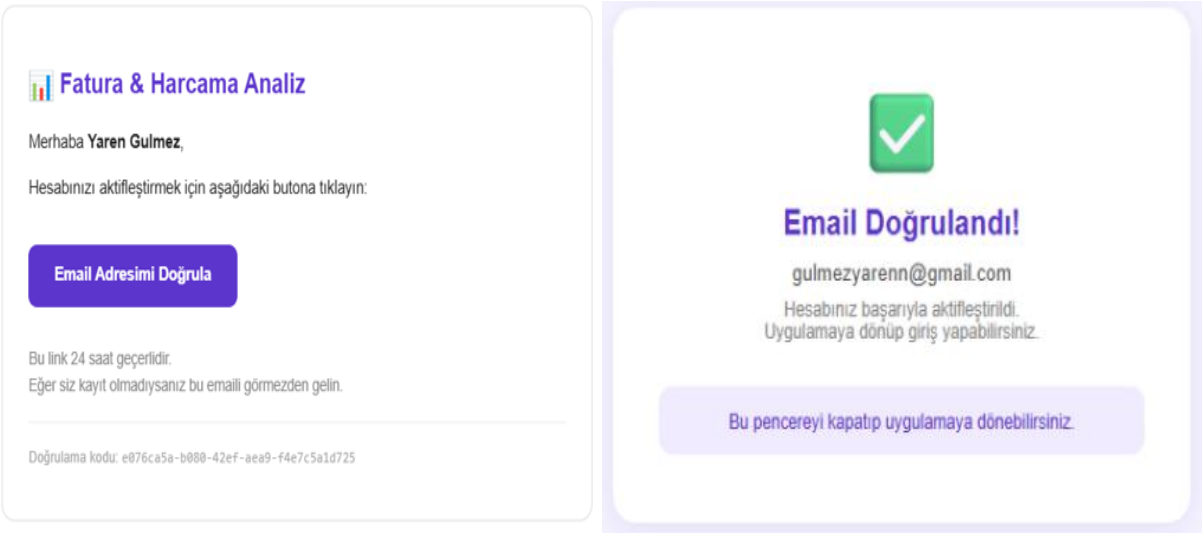


Figure 3.3.Password reset screen and OTP email delivered to the user

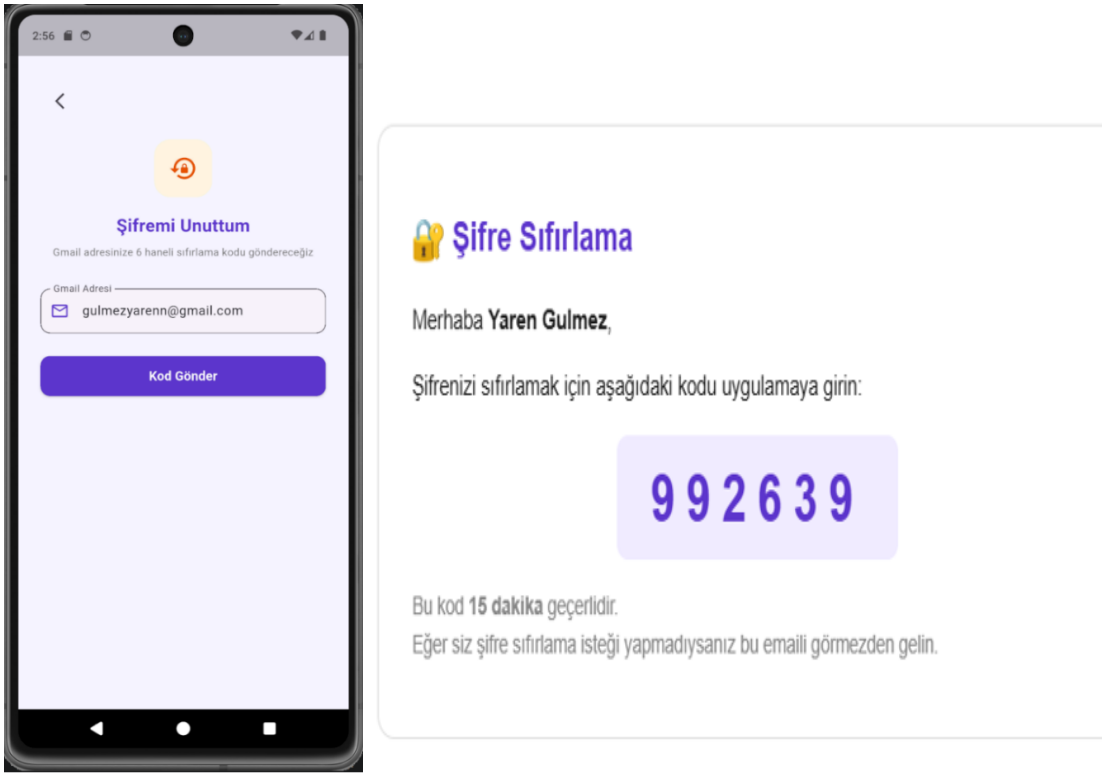
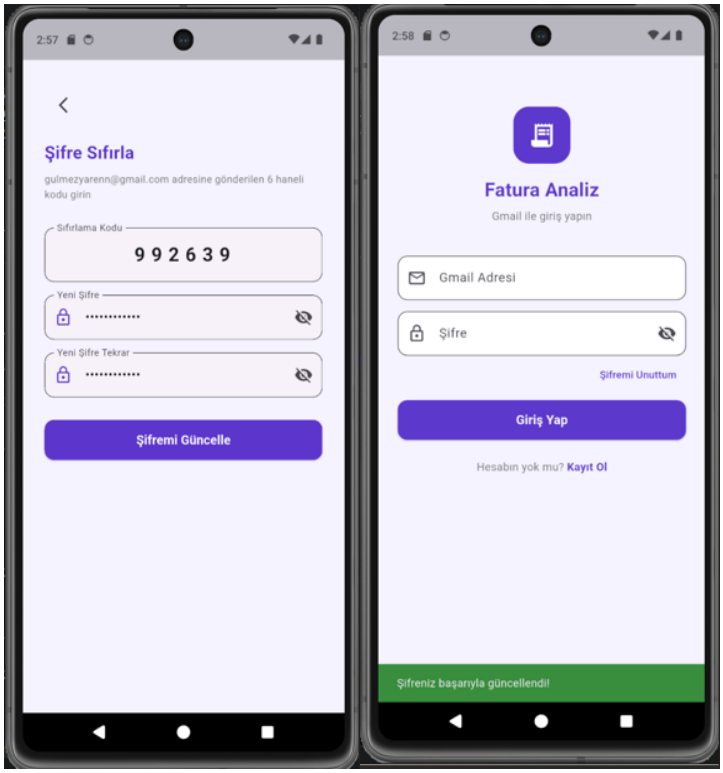


Figure 3.4.OTP entry and successful password reset confirmation screens

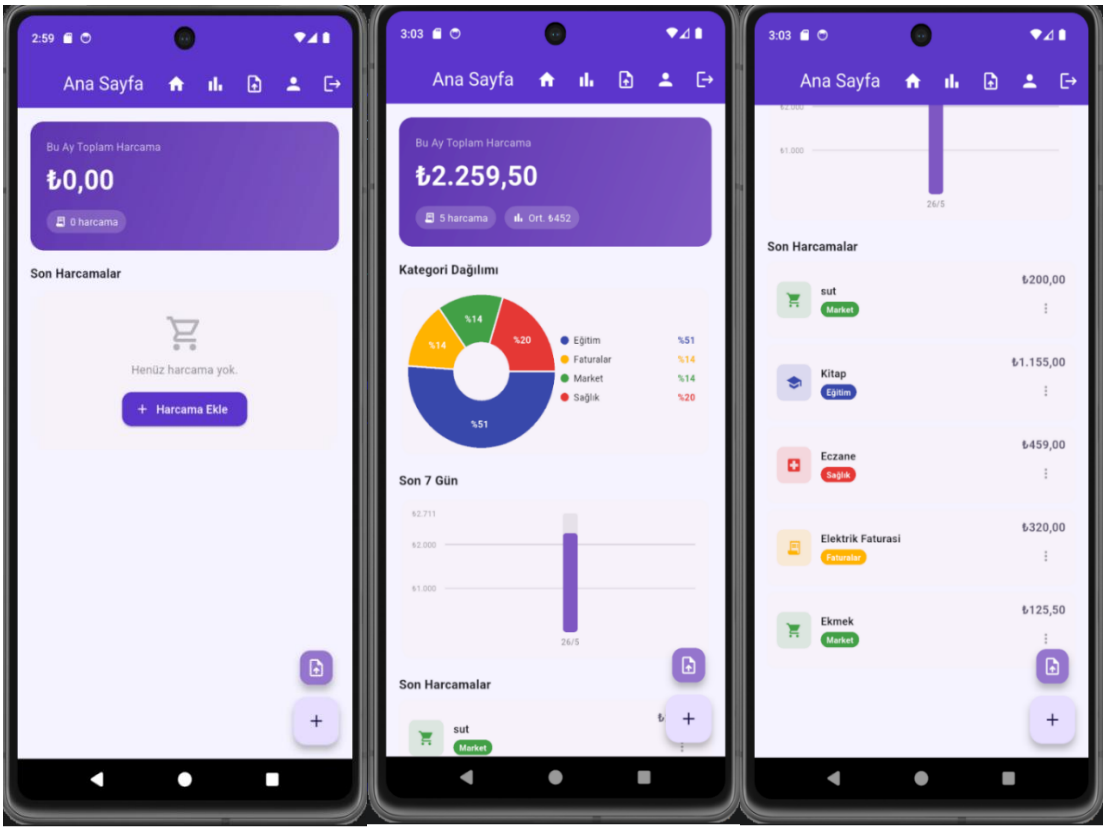


3.6.3. Home Screen

The home screen serves as the primary dashboard of the application. Its upper section displays a summary card showing the current month's total expense amount, the count of recorded expenses, and the daily expenditure average. Below this card, a donut chart presents the category distribution of all current-month expenses, with each category represented by a distinct color and percentage label. A bar chart immediately below visualizes the daily spending amounts for the last seven days. The lower section of the screen displays the most recent expense records in a scrollable list, with each item showing a category-specific icon, the expense name, a color-coded category badge, and the amount.

New expense records can be added through two access points: a floating action button in the lower right corner and an AI-assisted import button adjacent to it that navigates to the İçe Aktar screen. The top navigation bar provides access to the statistics screen, the import screen, the profile screen, and the logout action. Figure 3.5. shows the home screen in both its empty state and populated state with expense data and chart visualizations.

Figure 3.5.Home screen displaying monthly total, category distribution chart, and recent expenses

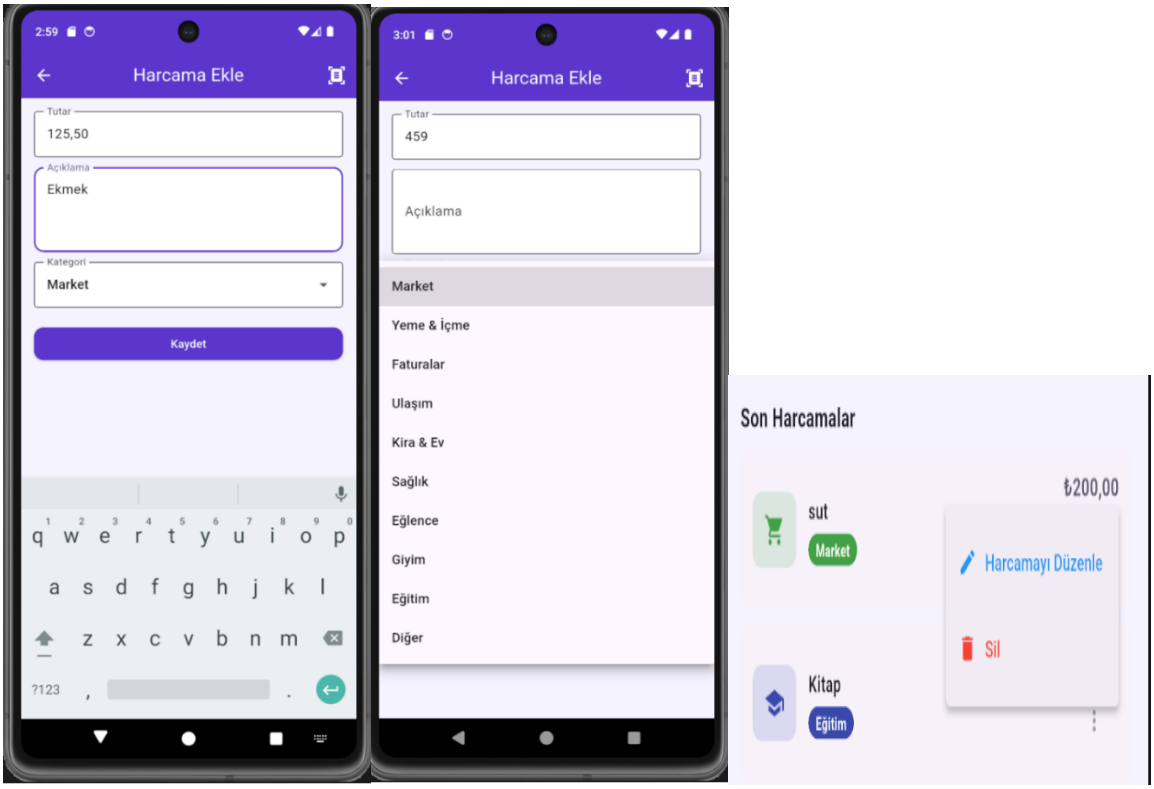


3.6.4. Expense Entry and Editing

The Harcama Ekle screen allows users to manually enter expense records. The form presents a Tutar field for the numeric amount, an Açıklama field for an optional free-text description, and a Kategori dropdown selector presenting all ten predefined categories. The Kaydet button submits the form to the backend and returns the user to the home screen upon success.

Existing expense records can be modified by tapping the three-dot context menu icon on any expense list item. This reveals a context menu presenting a "Harcamayı Düzenle" option and a "Sil" option with a delete icon. Selecting the edit option pre-populates the expense entry form with the existing record's values for modification. Figure 3.6. shows the expense entry screen, the category selector dropdown, and the context menu for edit and delete operations.

Figure 3.6.Expense entry screen showing amount input, category selection dropdown, and edit/delete actions



3.6.5. AI-Assisted Import Screen

The İçe Aktar screen is divided into two tabs: Dosya and Fotoğraf.

The **Dosya tab** supports batch import from CSV and XLSX files. An informational card specifies that the file must contain tutar and aciklama columns and describes the three-step workflow: select the file, trigger AI analysis, and save the results. The "Dosya Seç" button opens

the device file picker. After selection, the "Analiz Et" button submits the file to the backend for AI-powered category prediction. The results are presented as a list of predicted records showing the description, AI-assigned category, and amount. The user can review the full list and save all records with a single "İçe Aktar" action.

The **Fotoğraf tab** enables invoice image analysis. An informational card describes the three-step workflow: capture or select the image, the AI reads the image and predicts the category and amount, and the user reviews and saves the result. Two buttons allow the user to capture a new image using the device camera or select an existing image from the gallery. After image selection, the "Analiz Et" button submits the image to the Gemini 2.5 Flash microservice. The results are displayed in a preview section showing the total number of records analyzed, the number successfully processed, and the number skipped, followed by a "Tahmin Edilen Kategoriler" list presenting the predicted category and extracted amount for each detected expense. A "Tümünü Kaydet" button persists all predicted records to the database. Figure 3.7. and Figure3.8. shows the import screen for both file and photo modes, including an example analysis result for a petrol station receipt where the Gemini 2.5 Flash model correctly extracted an amount of 892.25 TL and assigned the Diğer category based on the receipt content.

Figure 3.7.AI-assisted import screen showing CSV/XLSX file import and photo import tabs

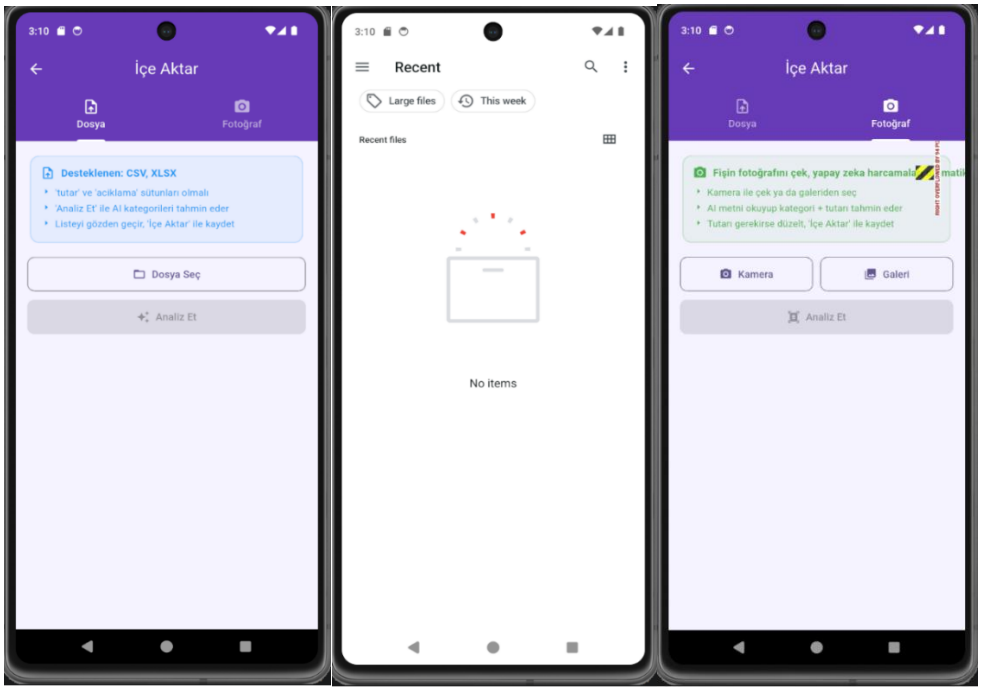
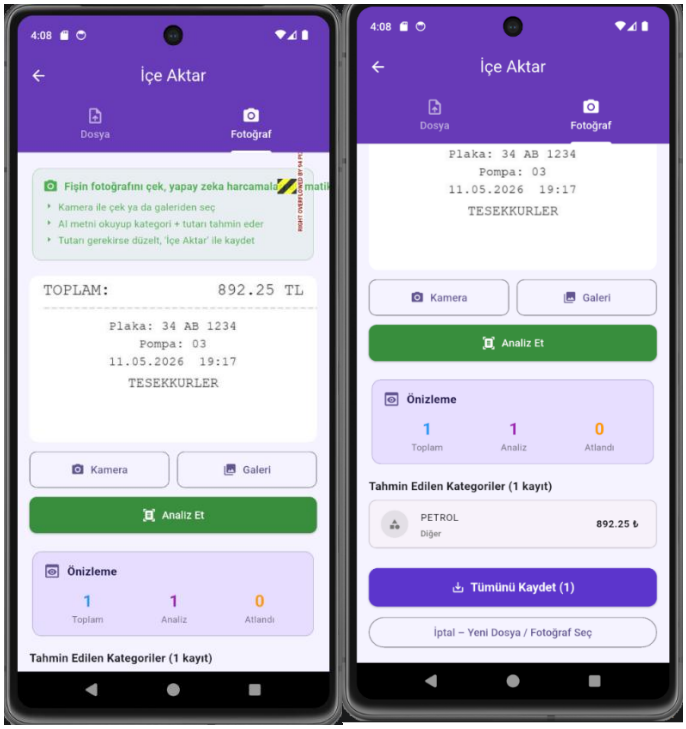


Figure 3.8.Invoice photo analysis result displaying extracted amount of 892.25 TL and predicted category.



3.6.6. Statistics Screen

The statistics screen is organized into four tabs: Özet, AI Analiz, Bütçe, and Aylık.

Özet Tab: The summary tab presents three key financial indicators in a card layout: Toplam Harcama (total expense amount), Günlük Ortalama (daily average expenditure), and En Çok Harcanan (highest-spending category). Below these indicators, a donut chart presents the category distribution with percentage labels displayed directly on the chart segments and a color-coded legend. A "Son 7 Gün Trendi" bar chart below the donut chart shows daily spending amounts for the seven most recent days, with the vertical axis labeled in Turkish Lira units. Figure 3.5 shows the Özet tab with active expense data.

AI Analiz Tab: The AI analysis tab presents Gemini-generated commentary organized into labeled sections. The "Yapay Zeka Yorumları" section displays individual insight cards covering the highest spending category, its proportional share of total expenditure, and the count of detected anomalous transactions. The "Anormal Harcamalar" section lists any statistically outlier expense entries with their amounts and a "Standart dış (istatistiksel)" label identifying them as statistically abnormal. A "Yüksek Risk" alert card, color-coded in red, presents a detailed breakdown of all categories flagged as potentially excessive along with the estimated monthly expenditure projection and specific per-category recommendations. The "Yapay Zeka

Önerileri" section displays an estimated monthly saving potential figure. Figure 3.6 shows the AI Analiz tab with anomaly detection output and risk assessment.

Bütçe Tab: The budget management tab allows users to define spending limits for each of the ten expense categories individually. Each category is presented as a card showing the category name and icon, the current month's actual spending, the defined budget target, and a progress bar visually representing the ratio of actual spending to the budget limit. Progress bars are color-coded green when spending is well within the budget, yellow when approaching the limit, and red when the limit is exceeded. Tapping the "Hedef" button on any category card opens a dialog allowing the user to define separate daily, weekly, and monthly budget limits for that specific category. Figure 3.7 shows the budget management tab with defined limits and progress indicators across multiple categories.

Aylık Tab: The monthly comparison tab presents a side-by-side comparison of the current month's and previous month's total expenditures, accompanied by an indicator describing the directional change. The "Yapay Zeka Yorumları" section below presents AI-generated commentary for each category that has changed between months, expressing percentage increases or decreases in natural language. The "Kategori Karşılaştırması" section lists each active category with the current month's amount alongside the previous month's amount and a percentage change indicator. Figure 3.9., 3.10. and 3.11. shows the Aylık tab with month-over-month comparison data and AI commentary.

Figure 3.9.AI analysis tab displaying Gemini-generated commentary, anomaly detection, and risk assessment.

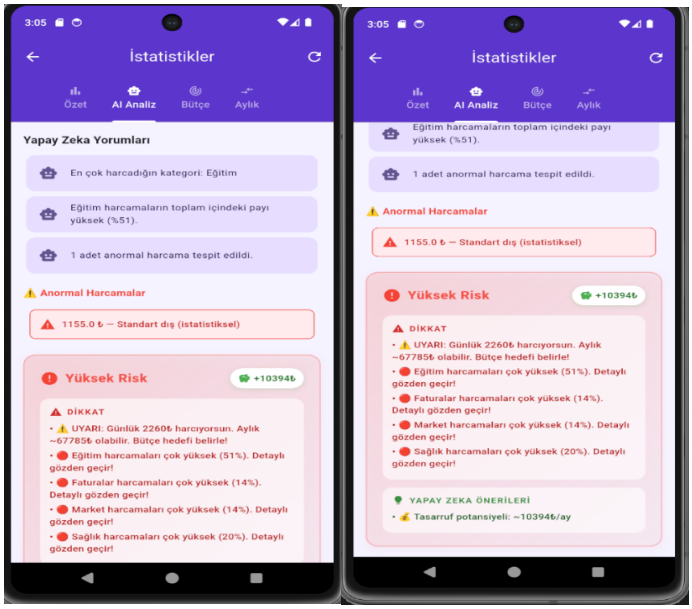


Figure 3.10.Budget management tab showing category budget goal dialog and color-coded spending progress indicators.

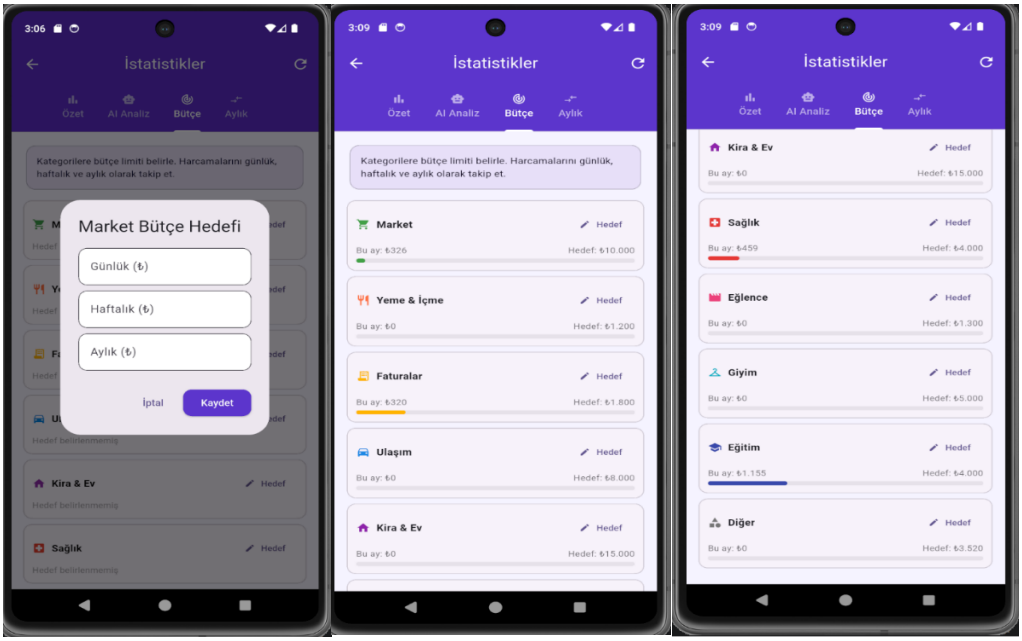
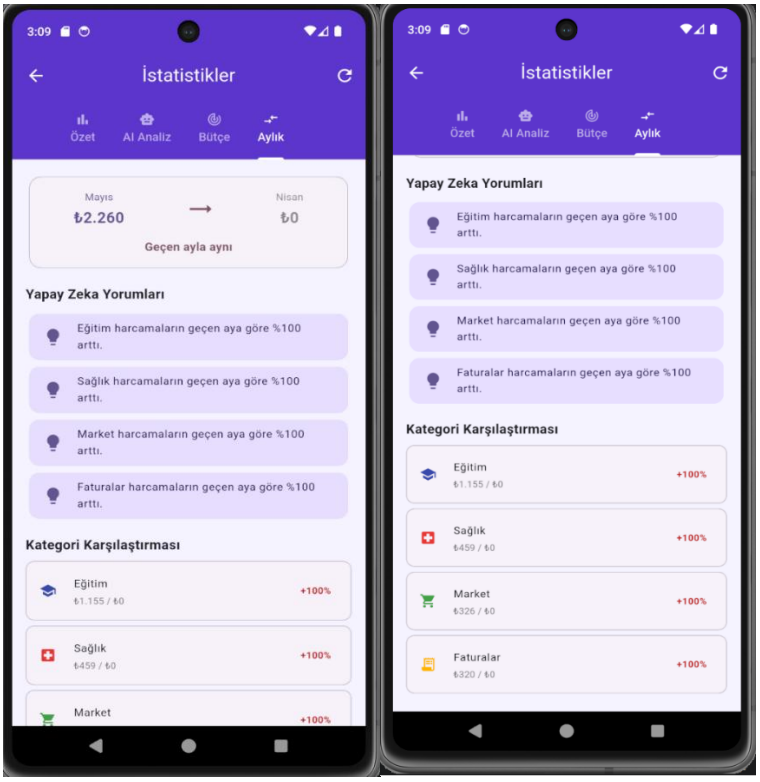


Figure 3.11.Monthly comparison tab displaying AI-generated commentary and category-level percentage changes



3.6.7. Profile Screen

The profile screen displays a user information card at the top containing the user's initials avatar, full name, email address, and an "Aktif Kullanıcı" status badge. Below the card, three summary statistics show the total expense amount, daily average, and active category count.

The **Ayarlar** section provides two configuration options. The **Karanlık Mod** toggle switches the application between light and dark visual themes; when dark mode is enabled, all application screens including the home screen charts and statistics tabs adopt a dark color scheme with consistent contrast ratios maintained across all UI elements. The **Dil** selector opens a dialog allowing the user to switch between Türkçe and English, applying the selected language immediately across all application strings without requiring a restart.

The **Hesap** section provides two account management actions. **Aylık Rapor İndir** generates a PDF document summarizing the current month's expenses, including the total amount, daily average, highest-spending category, and a category breakdown table showing each category's total expenditure and percentage share of the monthly total. The PDF is rendered with FaturaAnaliz branding and application version information and can be shared or printed directly

from the device. **Tüm Harcamaları Sil** permanently removes all expense records for the authenticated user after confirmation through a protected dialog.

The **Çıkış Yap** action displays a confirmation dialog before clearing the stored JWT and navigating back to the login screen. Figure 3.12. and 3.13. shows the profile screen in light and dark modes, the language selector dialog, the English-language profile view, and the monthly PDF report preview.

Figure 3.12.Profile screen in light and dark modes showing user statistics and application settings.

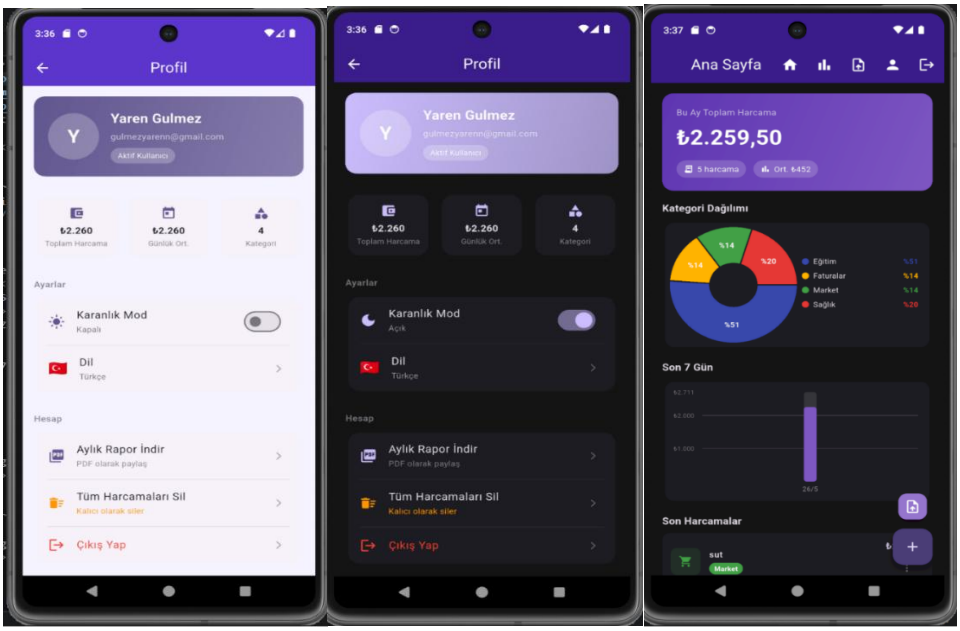
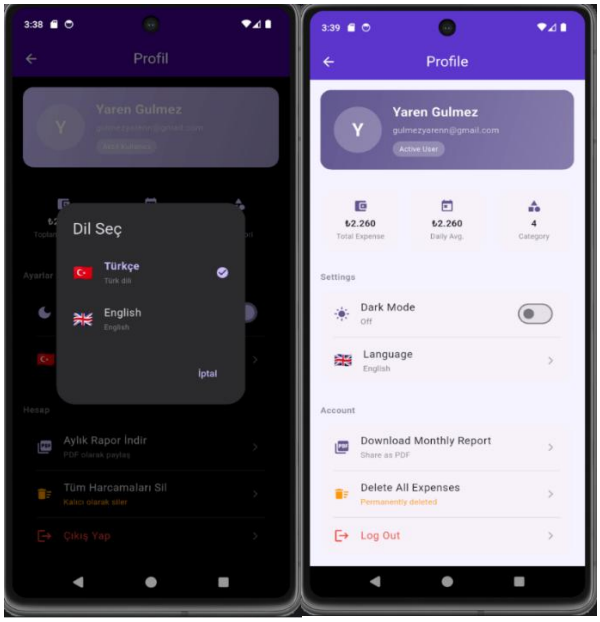


Figure 3.13.Language selection dialog and English-localized profile screen.



4. FINDINGS AND DISCUSSION

This chapter presents the findings obtained from the testing and evaluation of the developed AI-supported expense and invoice analysis system. The system was validated through a series of systematic tests applied across all functional layers, including backend API endpoints, the AI microservice integration, and the mobile application interface. The results of these tests are reported in the following sections, after which the system's capabilities and contributions are compared with related studies in the literature, and the findings are discussed in terms of their significance and implications.

4.1. Testing Strategy

The testing process was designed to validate all functional and non-functional requirements identified in Chapter 3. Given the multi-component architecture of the system — comprising the Spring Boot REST API backend, the Python Flask AI microservice, and the Flutter mobile client — the testing strategy was organized into three complementary layers: backend API testing, AI microservice integration testing, and mobile application testing. These were followed by an end-to-end user scenario validation that exercised all components in combination.

Backend API testing was performed by submitting HTTP requests to all defined endpoints using the Postman tool. Both successful request scenarios and intentionally erroneous or unauthorized requests were evaluated to confirm that the system behaves correctly under nominal and adverse conditions. AI microservice integration testing focused on verifying that the Gemini 2.5 Flash API responses were correctly parsed, that the category prediction pipeline operated reliably for both text and image inputs, and that statistical analysis requests produced structurally valid outputs. Mobile application testing was conducted manually on a physical Android device, following predefined user scenarios that covered the complete application workflow from account creation through AI-assisted expense analysis, budget management, and profile configuration.

4.2. Backend API Testing

4.2.1. Authentication and Email Verification Testing

The registration endpoint was tested with valid input containing a full name, Gmail address, password, and password confirmation. The system successfully created the user record with is_verified set to false and dispatched an HTML-formatted verification email to the provided

Gmail address. The email contained a clickable verification link and a plain-text token fallback. Upon accessing the verification link, the backend correctly set is_verified to true and returned the web confirmation page. Subsequent login attempts with the unverified account prior to clicking the verification link were correctly rejected with an appropriate error message, confirming that the email verification gate is enforced.

The forgot-password endpoint was tested by submitting a registered email address. The system generated a six-digit OTP, stored it alongside a fifteen-minute expiration timestamp, and dispatched a styled HTML email containing the code. The reset-password endpoint was then tested with the correct OTP and a new password, resulting in successful password update and clearance of the OTP fields. A second test submitted an expired OTP by waiting beyond the fifteen-minute window, and the system correctly rejected the reset attempt with a timeout error response.

The login endpoint was tested with correct credentials, returning a signed JWT containing the user's ID and email claims. Login attempts with incorrect passwords were rejected with HTTP 401 responses, and login attempts for unregistered email addresses returned appropriate error messages.

Figure 4.1.Successful authentication test via Swagger UI showing JWT generation.

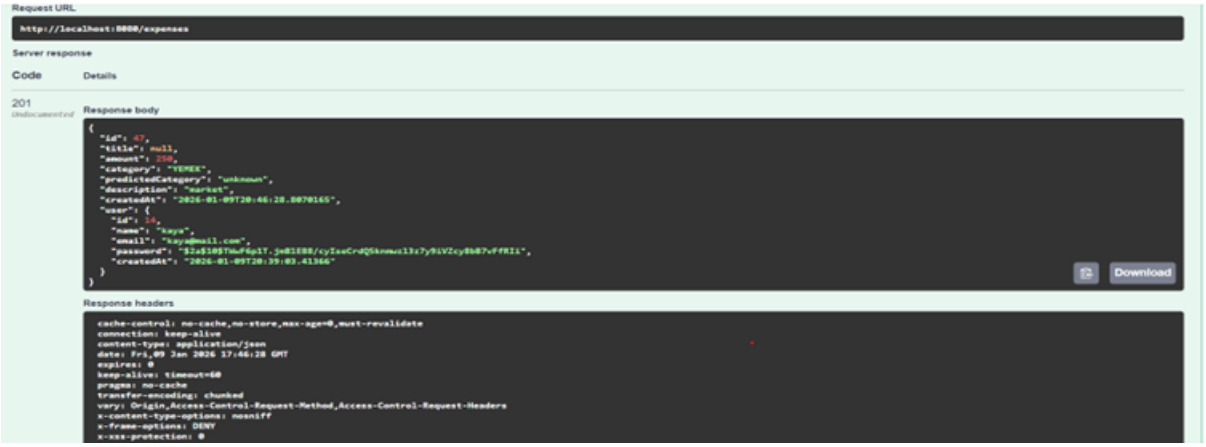


4.2.2. Expense Management Endpoint Testing

The expense creation endpoint was tested with a valid Bearer JWT in the Authorization header. Expense records with varying amounts, descriptions, and category values from the predefined ten-category set were successfully created and confirmed to be stored in the database with the correct user_id foreign key. The expense listing endpoint returned only the records belonging to the authenticated user, confirming user-based data isolation. Cross-user access was tested by substituting a JWT belonging to a second test user and verifying that only that user's records were returned.

The update endpoint correctly modified expense records identified by their database ID, and correctly rejected update attempts targeting records belonging to a different user with an HTTP 403 response. The delete endpoint removed the target record from the database and returned an HTTP 200 confirmation. The statistics aggregation endpoint returned the correct monthly total, daily average, category distribution percentages, anomaly flags for entries exceeding two standard deviations above the mean, and a formatted seven-day daily breakdown.

Figure 4.2.Successful expense creation test demonstrating user-based data isolation via Swagger UI.



4.2.3. AI Microservice Integration Testing

The invoice photo analysis endpoint was tested by submitting a base64-encoded petrol station receipt image. The Gemini 2.5 Flash model correctly extracted the total amount of 892.25 TL from the receipt text and assigned the Diğer category based on the PETROL description visible in the receipt content. The response was parsed correctly by the microservice and returned as a structured JSON object to the backend, which forwarded it to the mobile client. The full round-trip latency from image submission to result display was observed to remain within an acceptable range for a mobile application workflow.

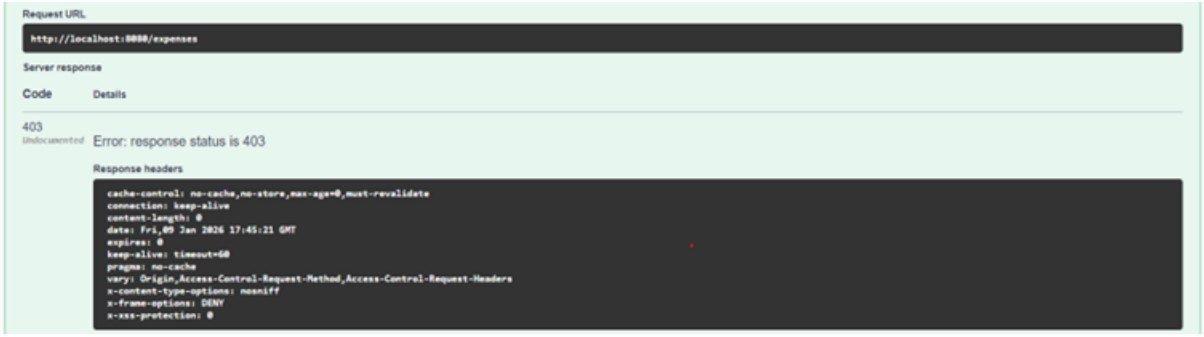
The CSV/XLSX batch import endpoint was tested with a file containing 20 expense records with Turkish-language descriptions spanning all ten predefined categories. The Gemini 2.5 Flash model correctly classified 17 out of 20 records, achieving an 85% classification accuracy on this test set. The three misclassified records contained ambiguous descriptions that could reasonably belong to multiple categories. The microservice returned a structured list of predicted expense objects, all of which were successfully batch-persisted to the database by the backend service.

The statistical analysis endpoint was tested by submitting an aggregated expense payload containing category totals, daily averages, and anomaly indicators. The microservice returned a structured JSON response containing Gemini-generated commentary in Turkish covering the highest spending category, detected anomalous entries, an overall risk assessment level, per-category recommendations, and an estimated saving potential figure. The response parsed correctly and all fields were rendered in the appropriate sections of the AI Analiz and Aylık tabs.

4.2.4. Security and Authorization Testing

All protected endpoints were tested with requests containing no Authorization header, returning HTTP 403 responses and confirming that unauthenticated access is blocked at the JWT filter level. Tests with syntactically malformed tokens and tokens signed with an incorrect key also resulted in correct rejection responses. The password hashing mechanism was verified by confirming that the stored password field in the PostgreSQL database contains a BCrypt hash rather than the plaintext value submitted during registration. These results confirm that the JWT-based security layer and BCrypt password storage meet the security requirements defined in Section 3.1.2.

Figure 4.3.Authorization test showing rejection of unauthorized access without a valid JWT token via Swagger UI.



4.3. Mobile Application Testing

4.3.1. Authentication Flow Testing

The complete authentication flow was exercised on a physical Android device. The registration screen accepted valid input and displayed a success notification informing the user that a verification email had been dispatched. The verification email was received in the Gmail inbox and the activation link correctly navigated the browser to the web confirmation page. After verification, login with the confirmed credentials succeeded and the application navigated to the home screen with the JWT stored locally.

The forgotten password flow was initiated from the login screen. The OTP email was received, and the six-digit code was entered on the reset screen alongside a new password. The application displayed the "Şifreniz başarıyla güncellendi!" toast confirmation and navigated back to the login screen. Login with the new password then succeeded correctly. The logout confirmation dialog was verified to clear the locally stored JWT and return the user to the login screen.

4.3.2. Expense Management Testing

The expense entry form was tested with valid amount, description, and category inputs. The record appeared immediately in the home screen expense list after successful submission. The three-dot context menu was accessed on an existing expense item, and both the edit and delete actions were verified. The edit action pre-populated the form with the existing record's values, and the updated record was correctly reflected in the list after saving. The delete action removed the record from both the interface and the database. The category dropdown was confirmed to display all ten predefined categories correctly.

4.3.3. AI-Assisted Import Testing

The Dosya tab of the import screen was tested by selecting a CSV file containing expense records with Turkish descriptions. After pressing Analiz Et, the AI-predicted categories were displayed in the results list. All predicted records were saved with a single tap on İçe Aktar, and the home screen correctly reflected the updated total and expanded category distribution.

The Fotoğraf tab was tested using a petrol station receipt image selected from the device gallery. After pressing Analiz Et, the preview section displayed a total of 1 record analyzed, 1 successfully processed, and 0 skipped. The predicted category Diğer and the extracted amount 892.25 TL were correctly displayed in the Tahmin Edilen Kategoriler list. Tapping Tümünü Kaydet persisted the record, and it appeared in the home screen expense list as PETROL under the Diğer category.

4.3.4. Statistics and Budget Testing

The Özet tab correctly displayed the current month's total expense, daily average, and highest-spending category across all test scenarios. The donut chart rendered category percentages accurately and the Son 7 Gün bar chart reflected the correct daily spending amounts. The AI Analiz tab displayed Gemini-generated commentary cards in the Yapay Zeka Yorumları section, identified the high-value PETROL expense as anomalous in the Anormal Harcamalar section, and rendered a high-risk alert card with category-level recommendations.

The Bütçe tab was tested by setting daily, weekly, and monthly limits for the Market, Faturalar, and Sağlık categories using the budget goal dialog. The progress bars updated to reflect actual spending against the defined limits, with color transitions from green to yellow and red observed as the spending approached and exceeded the limits respectively.

The Aylık tab displayed the current month's total alongside the previous month's total of ₺0, correctly computing +100% changes for all active categories. The Yapay Zeka Yorumları section presented Gemini-generated commentary for each changed category, and the Kategori Karşılaştırması table listed all active categories with their respective amounts and percentage change indicators.

4.3.5. Profile and Settings Testing

The profile screen correctly displayed the authenticated user's name, Gmail address, total expense amount, daily average, and active category count. The dark mode toggle was activated and all application screens correctly transitioned to the dark color scheme without any visual artifacts. The language selector dialog was opened and English was selected; all application strings immediately updated to English across all screens without requiring a restart. Switching back to Turkish restored the original language correctly.

The Aylık Rapor İndir action generated a PDF document containing the FaturaAnaliz branding, the current month label, total expense, daily average, active category count, and a category breakdown table with amounts and percentage shares. The PDF was successfully shared via the Android sharing interface. The Tüm Harcamaları Sil action presented a confirmation dialog before executing the deletion, and after confirmation all expense records were removed and the home screen returned to its empty state.

4.4. User Scenario Validation

A complete end-to-end user scenario was executed to validate the integrated operation of all system components. The scenario began with account registration, email verification, and first login. The user then manually added five expense records across four categories. The AI-assisted photo import flow was executed using a receipt image, adding a sixth record automatically. The statistics screen was opened and all four tabs — Özet, AI Analiz, Bütçe, and Aylık — were reviewed, confirming that the Gemini-generated commentary correctly reflected the updated expense data. Budget limits were defined for three categories. The monthly PDF report was generated and shared. Dark mode was toggled and the language was switched to English and back. Finally, the logout confirmation dialog was used to terminate the session.

All steps in the scenario were completed successfully without errors. The data displayed across all screens was consistent with the records stored in the PostgreSQL database, and the AI-generated commentary correctly reflected the actual expense distribution and anomalies present in the test dataset. These results confirm that all system components operate correctly in combination and that the system fulfills the functional requirements defined in Section 3.1.1 in their entirety.

4.5. Comparison with Literature

The system developed in this study addresses several gaps identified in the reviewed literature. Prior mobile expense tracking applications reported by Kim and Park demonstrated that real-time accessibility and convenient data entry improve structured financial behavior; the developed system extends this foundation by integrating AI-assisted import through both CSV files and invoice photography, significantly reducing the manual effort required to record expenses. The photo-based import feature directly addresses the problem of irregular receipt storage identified in the problem definition, as users can now photograph a receipt and have the total amount and category extracted automatically by the Gemini 2.5 Flash multimodal model.

Liu, Wang, and Li demonstrated that AI-based categorization produces more consistent results than manual methods, though high performance requires large and well-balanced training datasets. The developed system circumvents this limitation by leveraging the Gemini 2.5 Flash large language model, which applies broad pre-trained knowledge to expense categorization without requiring a domain-specific training dataset. This approach aligns with the architectural flexibility advocated by Hasan and Islam, who emphasized modular and extensible architectures as critical for long-term sustainability in financial software systems; the Python microservice design allows the underlying AI model to be replaced or upgraded independently of the core backend.

The budget management and monthly comparison features extend beyond the capabilities of most systems reviewed in the literature. Al-Sharif and Yaseen reported that graphical and categorical reporting improves financial transparency; the developed system provides this through the Özet tab donut chart and trend bar chart while additionally offering real-time budget progress indicators and AI-generated month-over-month commentary that contextualizes the raw comparison data. The anomaly detection mechanism, which flags statistically outlier expenses using a two standard deviation threshold, provides a quantitative basis for the AI commentary that is absent from purely rule-based notification systems.

The security architecture of the developed system, combining BCrypt password hashing, JWT-based stateless authentication, email verification as a pre-activation gate, and OTP-based password reset with a fifteen-minute expiry window, directly addresses the security concerns raised by Ferreira and Rodrigues regarding centralized storage of personal financial data. Ferreira and Rodrigues identified that systems lacking layered authentication and session isolation mechanisms are particularly vulnerable to unauthorized data access; the developed system mitigates these risks through user-based data isolation enforced at both the service layer and the database layer, ensuring that no user can access another user's financial records through any API endpoint.

Compared to commercial applications such as Mint and Wallet analyzed by Bourassa, the developed system provides transparent AI commentary that explicitly labels Gemini-generated insights as artificial intelligence output, addressing the opacity concern identified with black-box categorization logic in commercial tools. The inclusion of Turkish-language AI commentary and localization support for both Turkish and English addresses the usability and accessibility requirements identified by Singh and Kaur as determinants of long-term user engagement.

4.6. Limitations

Several limitations of the present study should be acknowledged. The AI categorization pipeline relies entirely on the Google Gemini 2.5 Flash API, which requires an active internet connection and introduces a dependency on an external commercial service. Beyond connectivity requirements, this dependency carries a long-term sustainability risk: the Gemini API operates under usage-based pricing and quota constraints, meaning that increased user load or changes in Google's pricing model could directly affect the operational viability of the system without architectural modifications. Latency in the Gemini API response also affects the user experience of the import and analysis features, and the quality of AI-generated commentary is inherently subject to the model's general-purpose training rather than domain-specific financial expertise.

The statistical anomaly detection method, based on a two standard deviation threshold applied to the user's own expense history, is effective only when a sufficient number of expense records have been accumulated. For users with few recorded expenses, the mean and standard deviation values may not be representative enough for reliable anomaly identification.

System testing was conducted using a single test account in a controlled development environment on a physical Android device. Long-term multi-user performance testing under

concurrent load conditions was not conducted within the scope of this study, and the system's behavior under high concurrent API request volumes has not been empirically evaluated. Furthermore, the application was tested exclusively on Android; iOS compatibility has not been verified, as cross-platform consistency was not evaluated beyond the single-platform development build.

These limitations identify clear directions for future work and do not undermine the primary research objectives achieved by the study.

.

5. CONCLUSION AND FUTURE WORK

This chapter presents the overall evaluation of the developed AI-supported expense and invoice analysis system, discusses its real-world applicability, and outlines directions for future research and development.

5.1. Conclusion

This study presented the design, implementation, and evaluation of a mobile-based expense and invoice management system integrating artificial intelligence capabilities through the Google Gemini 2.5 Flash large language model. The system was developed using a four-component layered architecture consisting of a Flutter mobile client, a Spring Boot 3 REST API backend, a PostgreSQL relational database, and a Python Flask AI microservice. All components were fully implemented and validated through systematic testing across backend API endpoints, AI microservice integration, and end-to-end mobile application scenarios.

The foundational implementation phase established the core infrastructure: user authentication, expense management, category-based visualization, and basic statistical reporting. The extended implementation phase substantially built upon this foundation by integrating the Gemini 2.5 Flash API across three distinct analysis modes. Invoice photo analysis enables users to capture or select a receipt image and have the total amount and expense category extracted automatically through Gemini's multimodal processing capability, eliminating the need for manual data entry from physical receipts. CSV and XLSX batch import allows users to import historical expense data with automatic AI-based category prediction applied to each record's description field. Statistical AI analysis generates Turkish-language natural language commentary covering spending behavior, anomaly detection based on statistical thresholds, risk assessment, and personalized saving recommendations, delivered across the AI Analiz and Aylık comparison tabs of the statistics screen.

Beyond the AI integration, the extended implementation phase added several features that substantially increase the practical utility of the application. The email verification mechanism ensures that only valid Gmail accounts can activate and use the system, reducing the risk of unauthorized account creation. The six-digit OTP-based password reset flow with a fifteen-minute expiry window provides a secure and user-friendly credential recovery mechanism. The budget management module allows users to define independent daily, weekly, and monthly

spending limits for each of the ten expense categories, with real-time color-coded progress indicators that make budget status immediately interpretable. The monthly comparison tab presents AI-generated commentary on category-level changes between consecutive months, supporting more informed long-term financial planning. Dark mode and Turkish/English language switching improve accessibility and user experience across diverse device and preference contexts. The monthly PDF report generation feature allows users to produce and share a structured summary of their current month's financial activity directly from the mobile application.

The testing and evaluation results presented in Chapter 4 confirmed that all defined functional and non-functional requirements were successfully met. Authentication flows including registration, email verification, login, and OTP-based password reset operated correctly under both nominal and error conditions. Expense management operations demonstrated correct user-based data isolation, with no cross-user data access possible through any API endpoint. The AI microservice correctly parsed Gemini API responses for both image and text inputs and returned structured data for all three analysis modes. The mobile application rendered all screens correctly on a physical Android device, and the end-to-end user scenario validation confirmed seamless integration across all system components.

Compared with related studies in the literature, the developed system advances beyond systems that plan AI integration as future work by delivering a fully operational AI-assisted expense management platform. The use of a large language model for both multimodal invoice analysis and natural language commentary generation represents a more flexible and scalable approach than training domain-specific classification models, as it eliminates the requirement for large labeled datasets while providing strong categorization and commentary quality across diverse expense types and receipt formats. The modular microservice architecture ensures that the AI layer can be independently upgraded, replaced, or scaled without requiring modifications to the core backend or the mobile client, fulfilling the extensibility and maintainability requirements established at the outset of the project.

In real-world application, the system provides a practical tool for individuals seeking to improve their financial awareness and spending discipline. The combination of manual expense entry, AI-assisted bulk import, automated receipt analysis, budget goal tracking, and monthly AI commentary covers the complete personal expense management lifecycle within a single mobile application. The Turkish-language AI commentary and localization support make the

system particularly relevant for Turkish-speaking users who have limited access to financially-oriented AI tools in their native language. The security architecture, combining BCrypt password hashing, JWT-based stateless authentication, and enforced email verification, ensures that personal financial data is protected against unauthorized access in compliance with the security requirements identified in the non-functional requirements analysis.

Overall, the study successfully achieved its primary objective of developing a functional, secure, and AI-supported mobile expense and invoice analysis system. The system demonstrates that large language model APIs such as Google Gemini 2.5 Flash can be effectively integrated into personal finance applications through a lightweight microservice architecture, delivering meaningful AI-assisted features within the latency constraints of a mobile user experience.

5.2. Future Work

Although the developed system fulfills its defined objectives and delivers a complete AI-supported expense management solution, several directions for future enhancement have been identified based on the study's findings and limitations.

Income Tracking Module. The system currently focuses exclusively on expense recording and analysis. The addition of an income tracking module would enable users to record salary, freelance, or other income entries and visualize net financial balance over time. Integrating income data into the AI analysis pipeline would allow the Gemini model to generate more contextually relevant commentary, such as savings rate calculations and income-to-expense ratio assessments, substantially increasing the system's value as a personal financial management tool.

Multi-Currency Support. The current system operates exclusively in Turkish Lira. Supporting multiple currencies with real-time or periodically updated exchange rates would make the application suitable for users who manage expenses across different countries or currencies, such as international students, remote workers, and frequent travelers. This enhancement would require currency conversion logic at both the data persistence and reporting layers, as well as updates to the AI analysis prompts to handle mixed-currency expense datasets correctly.

Bank API Integration. A significant source of friction in expense tracking applications is the requirement for manual data entry. Integration with open banking APIs or bank statement import functionality would allow the system to automatically retrieve transaction data from a

user's bank account, eliminating manual entry for the majority of expenses. The AI categorization pipeline is already designed to accept description text as input, making this a natural extension that would require primarily an additional data ingestion layer without changes to the core analysis infrastructure.

Domain-Specific AI Model Fine-Tuning. While the Gemini 2.5 Flash model provides strong general-purpose categorization and commentary capabilities, fine-tuning a smaller specialized model on a curated Turkish-language personal expense dataset could improve categorization accuracy for domain-specific edge cases and reduce API latency and cost dependency on an external commercial service. This would also enable fully offline AI analysis capability, removing the internet connectivity requirement currently imposed by the Gemini API integration.

Comprehensive User Study. The evaluation conducted in this study was limited to functional testing in a controlled environment using a single test account. A structured user study involving a representative sample of participants across varying demographic profiles, financial literacy levels, and smartphone usage patterns would provide empirical evidence regarding the system's usability, the perceived value of the AI-generated commentary, and the effectiveness of the budget management features in supporting real spending behavior change. Such a study would also enable the identification of usability issues not apparent in developer-led testing and would provide a more rigorous basis for comparing the system's impact against the manual expense tracking approaches documented in the literature.

6. REFERANCES

[1] M. I. R. Sabab, M. S. Islam, and M. S. Rahman, "eExpense: A smart expense tracking system," in *Proc. IEEE International Conference on Innovations in Science, Engineering and Technology (ICISET)*, Chittagong, Bangladesh, 2018, pp. 1–5.

[2] N. Ismail, A. R. Ahmad, and Z. Zainol, "Digital expense management: User requirements for automated financial tracking tools," *Journal of Information Systems and Technology Management*, vol. 18, pp. 1–14, 2021.

[3] A. Garcia and B. Claour, "Mobile bookkeeper: A personal financial management system with OCR-based receipt scanning," in *Proc. IEEE International Conference on Information Technology and Digital Applications (ICITDA)*, 2021, pp. 1–6.

[4] J. L. Santos, M. A. Ferreira, and P. R. Costa, "Voicordit: Voice-based daily expenditure recording using CNN and LSTM deep learning models," in *Proc. IEEE International Conference on Consumer Electronics (ICCE)*, Las Vegas, NV, USA, 2024, pp. 1–4.

[5] A. Sharma, R. Patel, and S. Mehta, "AI-powered personal finance management applications: A review of machine learning and NLP integration," *International Journal of Financial Technology*, vol. 6, no. 1, pp. 22–38, 2025.

[6] S. Shailendra, R. Kumar, and A. Singh, "Money map: A data-driven personal finance tracker with machine learning and dynamic visualization," in *Proc. IEEE International Conference on Knowledge Engineering and Communication Systems (ICKECS)*, 2025, pp. 1–5.

[7] M. I. R. et al., "WONGA: Machine learning-driven personal finance management with automatic expense categorization," in *Proc. IEEE International Conference on Emerging Trends in Engineering and Technology (INCET)*, Belgaum, India, 2023, pp. 1–6.

[8] A. Shaik, B. Reddy, and C. Rao, "Automatic expense tracking systems: A systematic exploration of machine learning-based categorization approaches," *Journal of Financial Data Science*, vol. 6, no. 2, pp. 45–59, 2024.

[9] S. García-Méndez, F. de Arriba-Pérez, A. González-Castaño, and J. A. Regueiro-Janeiro, "Automatic classification of banking transaction descriptions using NLP and SVM," *Expert Systems with Applications*, vol. 151, p. 113380, 2020.

[10] G. Toran, A. Pérez, and J. López, "Weakly supervised bank transaction classification using deep neural networks," in *Proc. IEEE International Conference on Data Mining (ICDM)*, 2023, pp. 1–8.

[11] M. Pavlinek, V. Podgorelec, and M. Hericko, "Resource-efficient few-shot LLM-based approaches for banking transaction categorization," *Applied Sciences*, vol. 13, no. 14, p. 8172, 2023.

[12] J. C. Dela Cruz, M. A. Santos, and R. P. Reyes, "Mobile expense tracker with Tesseract OCR v5 for receipt scanning," in *Proc. IEEE International Conference on Computer and Information Sciences (ICCOINS)*, 2023, pp. 1–6.

[13] N. Ismail, S. Abdullah, and M. Razali, "Automated receipt scanning using convolutional recurrent neural networks for expense tracking," *IEEE Access*, vol. 12, pp. 34521–34533, 2024.

[14] A. Abdallah, M. A. Kasem, and A. Hamdy, "ReceiptSense: A large-scale benchmark dataset for receipt understanding and question answering," *arXiv preprint*, arXiv:2403.08828, 2024. [Online]. Available: https://arxiv.org/abs/2403.08828

[15] Gemini Team et al., "Gemini: A family of highly capable multimodal models," *arXiv preprint*, arXiv:2312.11805, 2023. [Online]. Available: https://arxiv.org/abs/2312.11805

[16] Gemini Team et al., "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," *arXiv preprint*, arXiv:2403.05530, 2024. [Online]. Available: https://arxiv.org/abs/2403.05530

[17] Gemini Team et al., "Gemini 2.5: Pushing the frontier with advanced reasoning," *arXiv preprint*, arXiv:2503.19786, 2025. [Online]. Available: https://arxiv.org/abs/2503.19786

[18] M. Berghaus, P. Baumgartner, and R. Schimpe, "Benchmarking multimodal large language models on invoice processing with zero-shot prompting," *arXiv preprint*, arXiv:2502.04816, 2025. [Online]. Available: https://arxiv.org/abs/2502.04816

[19] B. Kovács, T. Nagy, and A. Horváth, "LLM-centric pipeline for information extraction from invoices," in *Proc. IEEE International Conference on Large Language and Vision Models (FLLM2024)*, 2024, pp. 1–6.

[20] A. Gómez and J. Sánchez, "Evaluating general-purpose large language models for structured information extraction from electricity invoices," *Applied Sciences*, vol. 16, no. 3, p. 1042, 2026.

[21] M. Li, X. Zhang, and Y. Chen, "JWT-based stateless authentication for scalable microservice architectures," *IEEE Transactions on Services Computing*, vol. 13, no. 4, pp. 712–724, 2020.

[22] A. Pangarkar, R. Desai, and S. Kulkarni, "Enhancing Flutter application performance and scalability through microservices architecture on AWS," in *Proc. IEEE International Conference on Communication and Signal Processing (ICCSP)*, Chennai, India, 2024, pp. 1–5.

[23] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. of California, Irvine, CA, USA, 2000.

[24] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519

[25] Pivotal Software, "Spring Boot reference documentation," https://docs.spring.io/spring-boot/docs/current/reference/html/, accessed May 2026.

[26] Google, "Flutter documentation," https://docs.flutter.dev, accessed May 2026.

[27] Pallets Projects, "Flask documentation, version 3.0," https://flask.palletsprojects.com, accessed May 2026.

[28] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conference (SciPy)*, Austin, TX, USA, 2010, pp. 56–61.

[29] P. Bhattacharyya, "BCrypt: A password hashing algorithm," in *Proc. IEEE International Conference on Intelligent Systems and Signal Processing (ISSP)*, Anand, India, 2013, pp. 1–4.

[30] OpenAPI Initiative, "OpenAPI Specification v3.0," https://swagger.io/specification/, accessed May 2026.

[31] Google, "Google Generative AI Python SDK documentation," https://ai.google.dev/gemini-api/docs, accessed May 2026.

[32] PostgreSQL Global Development Group, "PostgreSQL 16 documentation," https://www.postgresql.org/docs/16/, accessed May 2026.

CURRICULUM VITAE

PERSONAL INFORMATION

Full Name **:** Uğur Avşar

Date and Place of Birth **:** 25.07.2002 / İstanbul

Foreign Language **:** English

E-mail **:** uguravsar10@gmail.com

EDUCATIONAL BACKGROUND

Degree	Field	School / University	Graduation Year
Bachelor's Degree	Computer Engineering	Düzce University	—
High School	—	Abdülkadir Aksu Anatolian High School	2020